

The Rise of the
General Agents
(Builder)

The Rise of the
AI Agents and
Agent Skills
(Result)

Agent Factory: Building Digital Full-Time Equivalents (FTEs)

From Manual Coding to Spec-Driven Automation: Monetizing Knowledge through Agent-Building-Skills or Agent-Building-Agents

Version: 1.0

Zia Khan - MBA, MSE, MAC, MA, CPA, CMA

<https://www.linkedin.com/in/ziakhan/>

<https://www.facebook.com/ziakhan>

Presentation NotebookLM: <https://notebooklm.google.com/notebook/9a463eb6-90b4-4155-943f-bf60fc0696e8>

Agent Factory Textbook: <https://agentfactory.panaversity.org>

The Rise of the
Spec-Driven Dev
(Builder)

The Rise of the
Custom Agent
SDKs (Result)

The Agent Factory Thesis

“In the AI era, the most valuable companies won’t sell software—they’ll manufacture AI employees, powered by agents, specs, skills, MCP, autonomy and cloud-native technologies”

The 2026 AI Commercial Playbook

Turning Your AI Expertise into Revenue

This presentation serves as the "Business Roadmap" for your future. It transitions from **technical feasibility to market execution**, showing the audience how to turn these new AI protocols (Agents, Agents Skills, MCP, SDKs) into digital employees for profitable business.

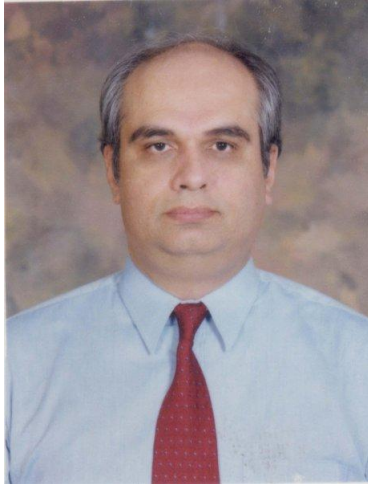


Start Building Your Digital Agent Factory Tonight

Once you understand this presentation, it positions you not just as an developer or educator, but as a strategic architect of the next phase of AI. We don't just teach people how to code; we are teaching them how to build and monetize Digital Agent Factories.

Zia Khan

Agentic AI Architect



Leadership

- CEO of Panaversity
- COO of PIAIC

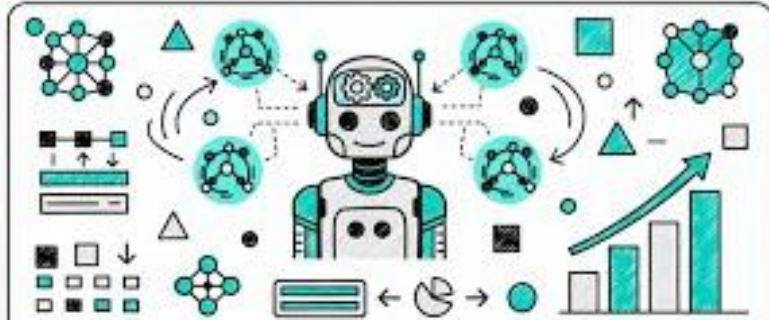
Impact & Credentials

- Founding educational AI startups
- Trained hundreds of thousands of students and professionals
- MBA, MSE, MAC from Arizona State University (ASU)
- CMA and CPA credentials (USA)

Nation-transforming AI education leader

Video Overviews

The Agentic AI Playbook



دی ایجنٹ فیکٹری



ChatGPT Has Changed The World

**Now Learn to Program in English or Urdu
instead of Python or TypeScript**



Claude Code

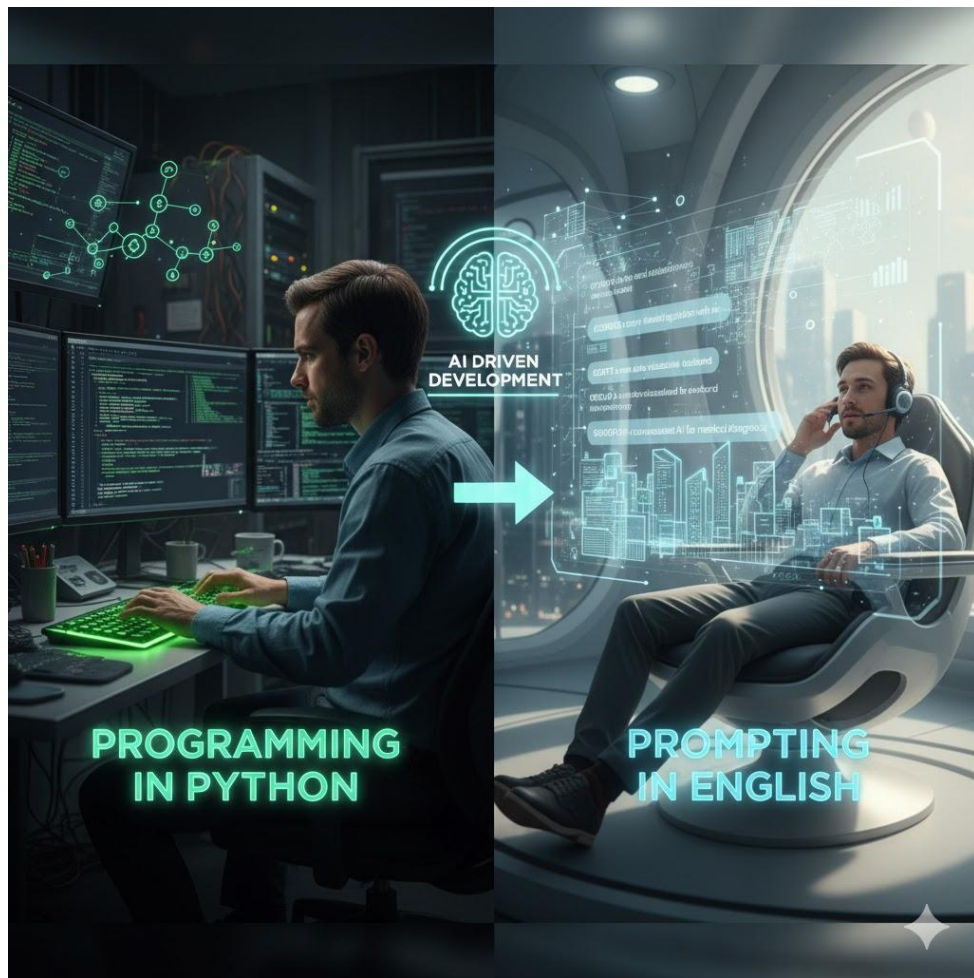


goose

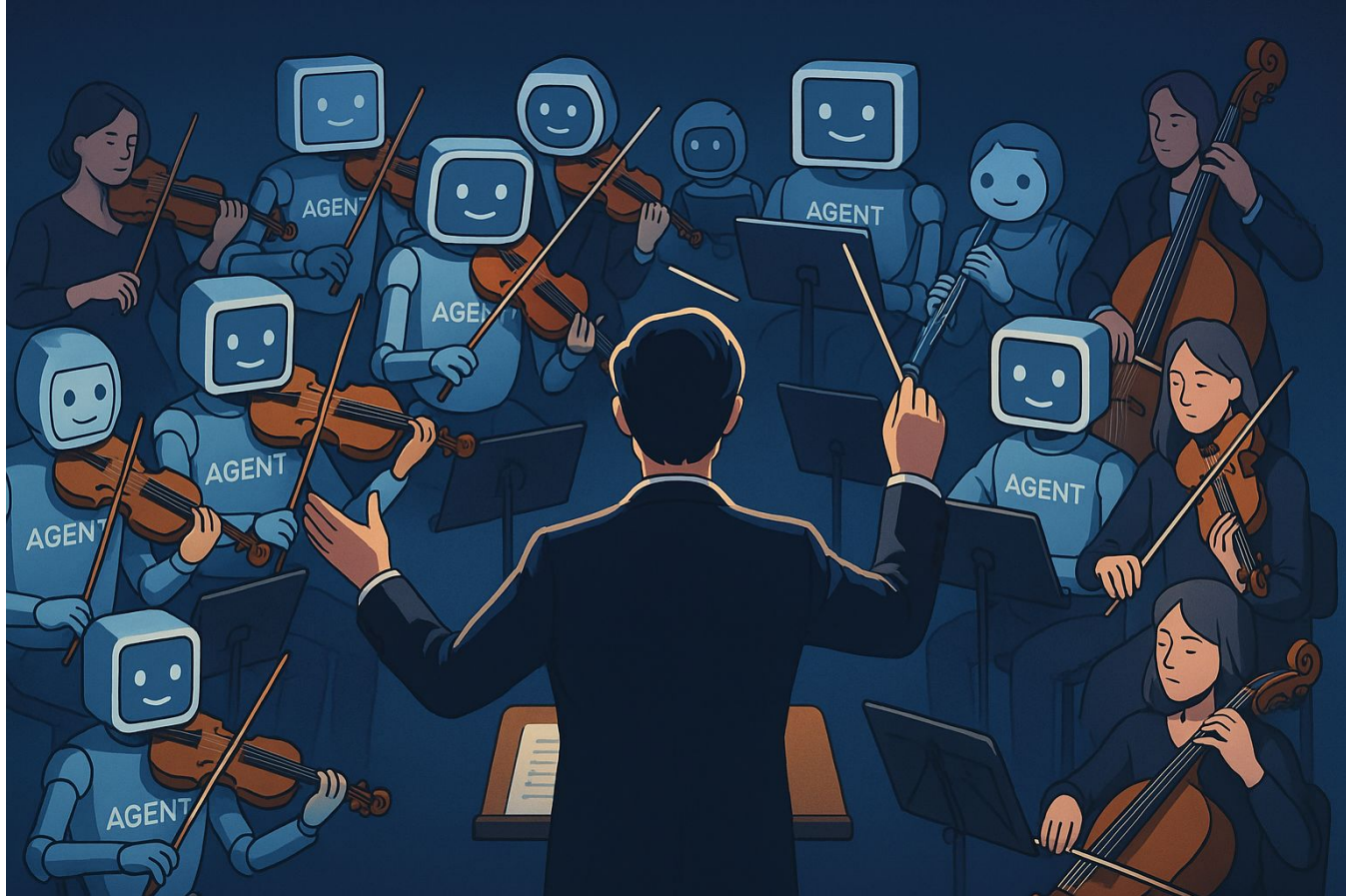
From User Interface to User Intent

The Age of Agentic AI is Here

From Python/Java/TypeScript... to natural-language-first



The shift from **developer-as-typist** to **developer-as-orchestrator**



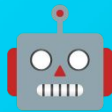
The Future of Work: A Partnership

Three forces working together



People

Judgment, creativity, oversight



Agents

Digital work automation

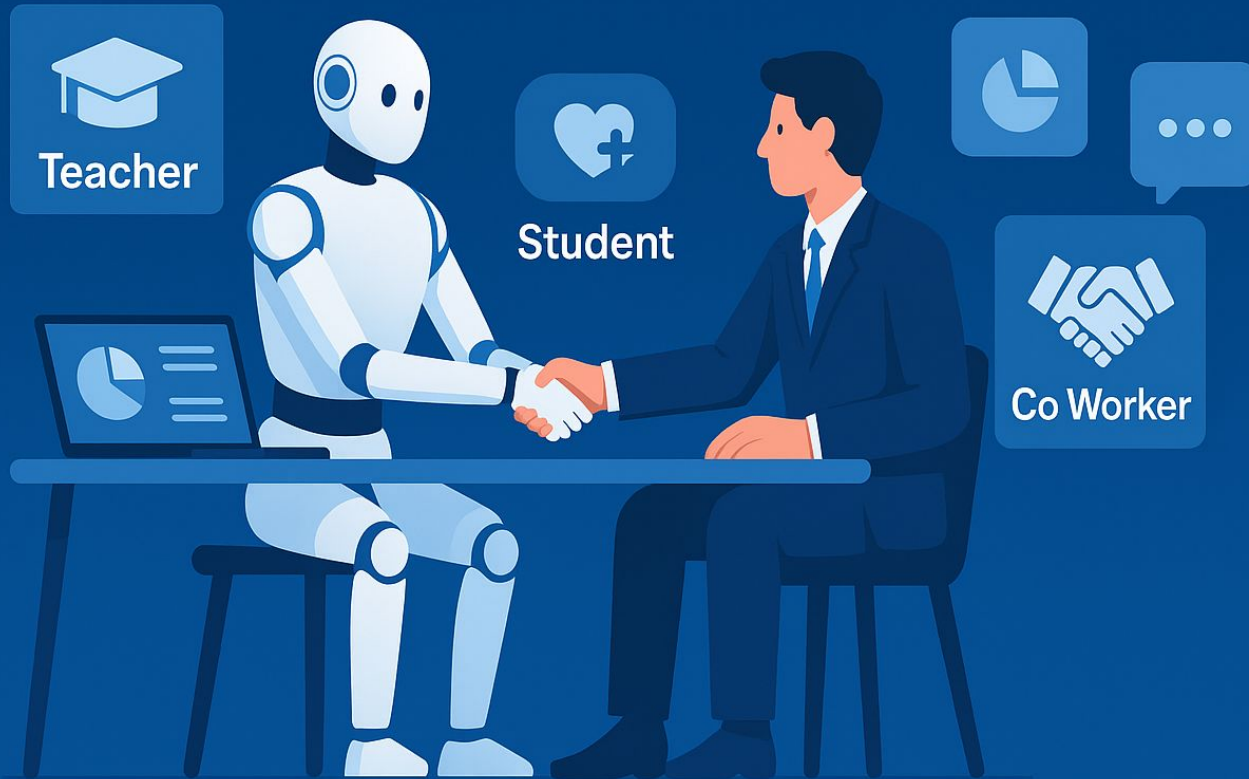


Robots

Physical work automation

AI is my Teacher, my Student, my Co Worker

Together, we will do everything.



The Three Waves of AI

Predictive AI

Focus on analyzing data to predict outcomes.

Analyze past data to predict future outcomes.

Why It Mattered: Enabled data-driven decision-making.

Generative AI

Focus on creating content from data.

Creating content (text, images, code, videos).

Why It Mattered: Empowered creativity and productivity.

Agentic AI

Focus on autonomous actions and learning iteratively.

Autonomous actions, environment interaction, iterative learning.

Why It Matters: AI becomes proactive, managing complex tasks.

What is an AI Agent?

An AI agent is a piece of software that can pursue a goal by observing its environment, deciding what to do next, taking actions (often by calling tools/APIs or controlling a robot), and learning from the results—then repeating the loop until the goal is met.

What makes it an “agent” (not just a chatbot)?

Goal-driven: You give it an objective (“pull the Xero trial balance daily and export CSV”), not just a single prompt.

Tool use / actions: It can call functions, APIs, databases, browsers, or devices—not only generate text.

State & memory: It keeps context (short-term working state and longer-term memory) across steps.

Autonomy: It plans multi-step work, executes, checks results, and adjusts without you micromanaging.

The Next Leap in AI

Moving from understanding to action



Large Language Models

AI that responds

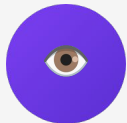


Large Action Models

AI that acts, orchestrates, and remembers

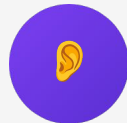
Autonomous Agents: The Five Powers

Systems that can see, hear, reason, act, and remember



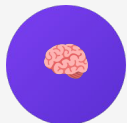
See

Visual understanding



Hear

Audio processing



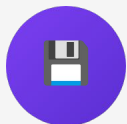
Reason

Complex decision-making



Act

Execute and orchestrate



Remember

Maintain context and learn

Redefining Everything

How agentic AI transforms our world



How We Work

AI agents as collaborative teammates



How We Transact

Autonomous systems managing complex transactions



How We Build

AI-first development from high-level intent

2025 Year of Agents

FEB

ANTHROPIC
How We Build Effective Agents

Barry Zhang

ANTHROPIC

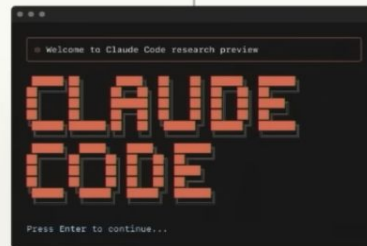
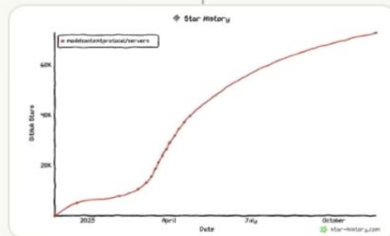
Model Context Protocol

Integrating LLMs with your tools & data

Mohesh Murug



Model Context Protocol



Claude Agent SDK



Agentic AI Foundation



OpenAI Agents SDK

The Core Strategic Decision

Three Paths to Agentic AI Automation



Option A

"Smart Consultant"

General Agents

Claude Code, Cowork, Codex, Goose

Session-based, autonomous reasoning. A senior contractor you bring in for a specific problem.

BUILD



Option B

"Assembly Line"

Custom-Built AI Employees

OpenAI Agents SDK, Google ADK, Claude Agent SDK

You architect every workflow, guardrail, and hand-off. You write the job description and training manual.

BUY



Option C

"Pre-Trained New Hire"

Pre-Built AI Employees

OpenClaw, Manus, Devin (Coding), Harvey (Law)

Pre-packaged, always-on, multi-channel. They come pre-trained; you just onboard them to your systems.

Options B & C are both **Digital FTEs (AI Employees)** — distinguished by the classic **Build vs. Buy** decision

General Agents

Claude Code, Cowork, Codex, Goose

The "Smart Consultant" — You bring them in for a specific problem

Key Features

- ✓ **zero-Shot Planning:** You state the goal; it determines the steps.
- ✓ **Deep Integration:** Accesses local files, git history, and command line directly.
- ✓ **Enhanced by MCP:** Uses "Model Context Protocol" to plug in external systems instantly.
- ✓ **Enhanced by Agent Skills:** Modular capabilities (organized folders with instructions, scripts, and resources).

Best For

- ✓ Complex debugging & coding
- ✓ Ad-hoc analysis (e.g., "Why are sales down in Q3?")
- ✓ Tasks requiring human-like judgment
- ✓ Expensive per task but invaluable for non-routine work

AI Employees: The Build vs. Buy Decision

Both paths produce Digital FTEs — the question is how you create them



BUILD — Custom-Built AI Employees

You are the architect.

You define every guardrail, hand-off, and orchestration step. The AI follows your script precisely.

Like writing a job description, SOPs, and training manual — then hiring a robot to execute them perfectly.



BUY — Pre-Built AI Employees

You are the manager.

They come pre-trained with capabilities. You onboard them to your systems, give them access, and set expectations.

Like hiring a skilled employee who already knows their craft — you just show them where the office is.

Custom-Built AI Employees

BUILD

OpenAI Agents SDK, Google ADK, Claude Agent SDK

The "Assembly Line" — You architect the workflow from scratch

Key Features

- ✓ **Guardrails:** Strict control over what the agent can and cannot do.
- ✓ **Orchestration:** Define exact hand-offs between multiple agents (e.g., Triage Agent → Support Agent).
- ✓ **UI/UX Flexibility:** Can be embedded in web apps, Slack, or internal dashboards.
- ✓ **Enhanced by MCP and Agent Skills:** Plug in external systems and modular capabilities.

Best For

- ✓ Standard Operating Procedures (SOPs)
- ✓ High-volume tasks (e.g., processing 5,000 invoices)
- ✓ Customer-facing interactions (requires strict safety)
- ✓ You are the architect. The AI follows the script.

Pre-Built AI Employees

[BUY](#)

OpenClaw, Manus

The "Pre-Trained New Hire" — Deploy, configure, and onboard

Key Features

- ✓ **Always-On:** Runs 24/7 on your infrastructure — persistent, proactive, autonomous.
- ✓ **Multi-Channel:** WhatsApp, Telegram, Slack, Discord, Teams — wherever you are.
- ✓ **Persistent Memory:** Remembers context across conversations and sessions.
- ✓ **Skill-Extensible:** 100+ AgentSkills, self-hackable, community-driven plugins.

Best For

- ✓ Personal productivity automation (email, calendar, task management)
- ✓ Cross-platform life/work orchestration from your phone
- ✓ DevOps monitoring, cron jobs, and proactive alerting
- ✓ A pre-trained employee you onboard — not build from scratch

Two Flavors of Pre-Built AI Employees

Option C comes in two varieties — distinguished by scope, not architecture

C₁ HORIZONTAL

General-Purpose AI Employees

They do everything across your life and work

- ✓ **OpenClaw:** Open-source, self-hosted, 150K+ stars, 100+ AgentSkills, multi-channel (WhatsApp, Slack, Telegram)
- ✓ **Manus:** Cloud-based autonomous agent, operates independently across tasks
- ✓ **Lindy AI:** No-code AI employees for email, scheduling, sales, support — prebuilt and ready to deploy

Scope: Wide — personal productivity, cross-platform orchestration, life/work automation

C₂ VERTICAL

Domain-Specific AI Employees

Pre-trained experts in a single professional domain

- ✓ **Devin (Coding):** Autonomous AI software engineer. Tag @Devin in Slack/Linear — it plans, codes, tests, and ships PRs.
- ✓ **Harvey (Law):** AI legal employee used by 50%+ of AmLaw 100. Researches, drafts, and analyzes across 60+ jurisdictions.
- ✓ **Emerging...:** Medicine, accounting, finance, HR — every profession will have its vertical AI employee.

Scope: Deep — mastery of one domain's data, workflows, regulations, and professional standards

Deep Dive on OpenClaw

Why it's unique?

Uniqueness 1

It's "Headless"

Unlike ChatGPT, which waits for a prompt in a browser, OpenClaw runs in the background (terminal/server).

Uniqueness 2

It's "Permissive"

It connects to real tools immediately (File System, Terminal, Slack/WhatsApp).

Uniqueness 3

The "Employee" Illusion

It feels like an employee because it persists. If you tell it "Monitor this server and fix it if it crashes," it stays awake and does it. A General Agent would time out; a Custom Agent would require you to write the monitoring loop code. OpenClaw provides the runtime for this "employment."

Three Paths Compared

Consultants vs. Custom-Built Employees vs. Pre-Built Employees

Dimension	Option A "Smart Consultant"	Option B BUILD Custom-Built AI Employee	Option C BUY Pre-Built AI Employee
Examples	Claude Code, Cowork, Codex, Goose	OpenAI Agents SDK, Google ADK, Claude Agent SDK	OpenClaw, Manus, Devin (Coding), Harvey (Law)
What It Is	Autonomous agent in terminal	Framework for building AI workflows	Pre-packaged AI employee
Your Role	State the goal	Architect the workflow	Configure & onboard
Operational Mode	Session-based, on-demand	Embedded in apps	Always-on, 24/7
Interface	Terminal / CLI	Custom UI / API	WhatsApp, Slack, Telegram
Memory	Per-session context	State managed by developer	Persistent across sessions
Workforce Analogy	Contractor	Custom-trained new hire	Pre-trained new hire
Best For	Complex, novel problems	SOPs, high-volume, compliance	Life/work automation

The Digital FTE Spectrum

How the Three Paths Map to Your Workforce

A

Contractor

B

Custom-Built FTE

C

Pre-Built FTE

B + C = Digital FTEs (AI Employees) — Build vs. Buy

You bring them in for a specific problem.

They reason through it, deliver the result, and leave.

*High cost per task.
High value for novel work.*

You design the workflow. They execute it perfectly.

Strict guardrails. Predictable output. Scales to thousands.

*Low cost per task.
Maximum control.*

You onboard them like a new hire. They show up every day.

Persistent memory, multi-channel presence, proactive behavior.

*Always available.
Fastest time to value.*

When to Use What

A Decision Framework for Agentic AI

Choose A: Consultant

- ✓ The task is novel or ambiguous
- ✓ You need deep code reasoning
- ✓ One-off complex analysis
- ✓ You want the agent to plan its own approach

Choose B: Build

- ✓ You need strict guardrails & compliance
- ✓ Processing thousands of items at scale
- ✓ Customer-facing interactions
- ✓ Predictable, repeatable output is critical

Choose C: Buy

- ✓ You want an always-on assistant fast
- ✓ Cross-platform life/work automation
- ✓ You prefer to configure, not code
- ✓ Persistent memory & proactive action matter

The Future is Hybrid

The most effective agentic organizations won't choose just one path.

General Agents for novel thinking

Custom-Built AI Employees for reliable, high-volume processes

Pre-Built AI Employees for persistent, always-on presence

Together, they form your AI-native workforce — your Digital FTEs.

The "Trojan Horse" of AI

Why "Claude Code" is Actually a General Agent. Moving Beyond the Name: Code as a Mechanism, Not a Constraint

- **The Misconception:** The name implies it is a tool strictly for software engineers—just a syntax highlighter or autocomplete.
- **The Reality:** It is an **Autonomous Problem Solver** that happens to speak the language of code.
- **The Distinction:**
 - **Coding Agents (e.g., Cursor):** Role-bound to software development workflows.
 - **General Agents (Claude Code):** Solve problems across any domain using code as the universal interface

"Don't let the name fool you. Calling it a 'Coding Agent' is like calling a CEO an 'Email Writer' just because they use email to do their job. Code is simply the tool it uses to exert power over the computer "

Authority Defines the Scope of Action

Where does the Agent live and what problems can it solve?

Feature	Coding Agent (e.g., Cursor)	General Agent (Claude Code)
Scope	Software development	Any business domain
Identity	Developer's pair programmer	Digital employee
Habitat	Embedded in developer tooling	Operates across system-level tools
Built For	Developers	Anyone solving problems
Example Tasks	"Implement feature, Refactor this module", "Write tests"	"Plan your 2026", "Draft emails", "Why did sales drop?"

Coding agents like Cursor are powerful, software-native agents optimized for development workflows.

General agents like Claude Code are goal-native agents trusted with broader objectives, able to choose tools, cross domains, and act at the system level using code as the universal interface.

The Cognitive Leap

From "Prediction" to "Reasoning"

- **Early Traditional Coding Agents (Predictive-centric):**
 - **Logic:** "Based on the last 10 lines, what is the most likely next line?"
 - **Limit:** They struggle to recover from errors without explicit guidance.
- **General Agents (Reasoning Loop):**
 - **Logic:** Uses an **OODA Loop** (Observe, Orient, Decide, Act).
 - **The Workflow:**
 1. **Observe:** "I see an error in the logs."
 2. **Decide:** "I will check if the Docker container is running."
 3. **Act:** *Executes Docker command.*
 4. **Correct:** "That didn't work, let me try a different flag."

"Early coding agents are primarily prediction-centric. General agents reason through problems. They enter a loop of thinking—acting, checking the result, and self-correcting. This is what allows them to handle complex tasks without you holding their hand."

Code is the Universal Interface

Why Business Questions Get Code Answers

- **The Paradigm Shift:** We don't just write code to build software; we use code to interrogate reality.
- **Example: "Why did sales drop in Q3?"**
 - **A Coding Agent:** Would confuse this for a code comment.
 - **A General Agent:**
 1. Writes a **SQL query** to fetch sales data.
 2. Writes a **Python script** to visualize the trend.
 3. Analyzes the chart.
 4. **Answer:** "Sales dropped because of 40% churn in the Enterprise sector."

"This is the core of our strategy. The General Agent acts as a Business Analyst. It translates a human question into a code execution to get a factual answer."

Infinite Extensibility via MCP and Agent Skills

Breaking the "Coder" Stereotype

- **The Enabler: Model Context Protocol (MCP) and Agent Skills.**
- **How it expands the role:**
 - Plug in **Slack MCP and Skill** → Becomes a **Communications Manager**.
 - Plug in **Salesforce MCP and Skill** → Becomes a **RevOps Specialist**.
 - Plug in **Xero/QuickBooks MCP and Skill** → Becomes a **Financial Auditor**.
- **The Verdict:** A strictly defined "Coding Agent" cannot send messages or audit finances. A General Agent can, provided you give it the right "hands" (MCP) and "procedures" (Agent Skill).

"This is why we classify it as 'General.' If you plug in a Finance MCP and Skill, it's a Finance Agent. If you plug in a Sales MCP and Skill, it's a Sales Agent. Code is just the glue that holds it all together."

Skills and MCP Combination

Skills are "Expertise Packs" (instructions + logic) while MCP is the "Data Pipe" (connectivity)

Agent Skills

The "How-To"

These are modular folders (with a SKILL.md) that teach Claude a specific, repeatable workflow (e.g., "Analyze this financial statement according to our company's Q4 risk framework"). It's about **standardizing expertise**.

MCP

The "With-What"

This is the protocol that connects those skills to your live data (e.g., your SQL database or Jira)

Agent Factory

"Welcome. We are entering the era of the 'Agent Factory.' While the industry talks about General Agents and Custom SDKs as separate trends, we are going to look at how they converge. As you'll see that **Code is the Universal Interface** that allows a General Agent to transform your knowledge into a deployed **Custom Skill or Custom Agent.**"



Enterprise Architecture Shift

From SaaS Tools to Digital FTEs

Before: Tool-Centric Enterprise

- Humans operate SaaS tools
- Logic lives in people's heads
- Automation is brittle and task-level
- Knowledge is undocumented and unscalable

Humans → SaaS Apps → APIs → Data

After: Agent-Centric Enterprise

- Humans manage outcomes
- Logic lives in Specs & Skills
- Automation is goal-driven
- Knowledge is reusable IP

Humans → Digital FTEs (Agents) →
Agent Skills (SKILL.md) + MCP →
Enterprise Systems & Data

Key Shift:

From *tools you use* → digital *teammates you manage*

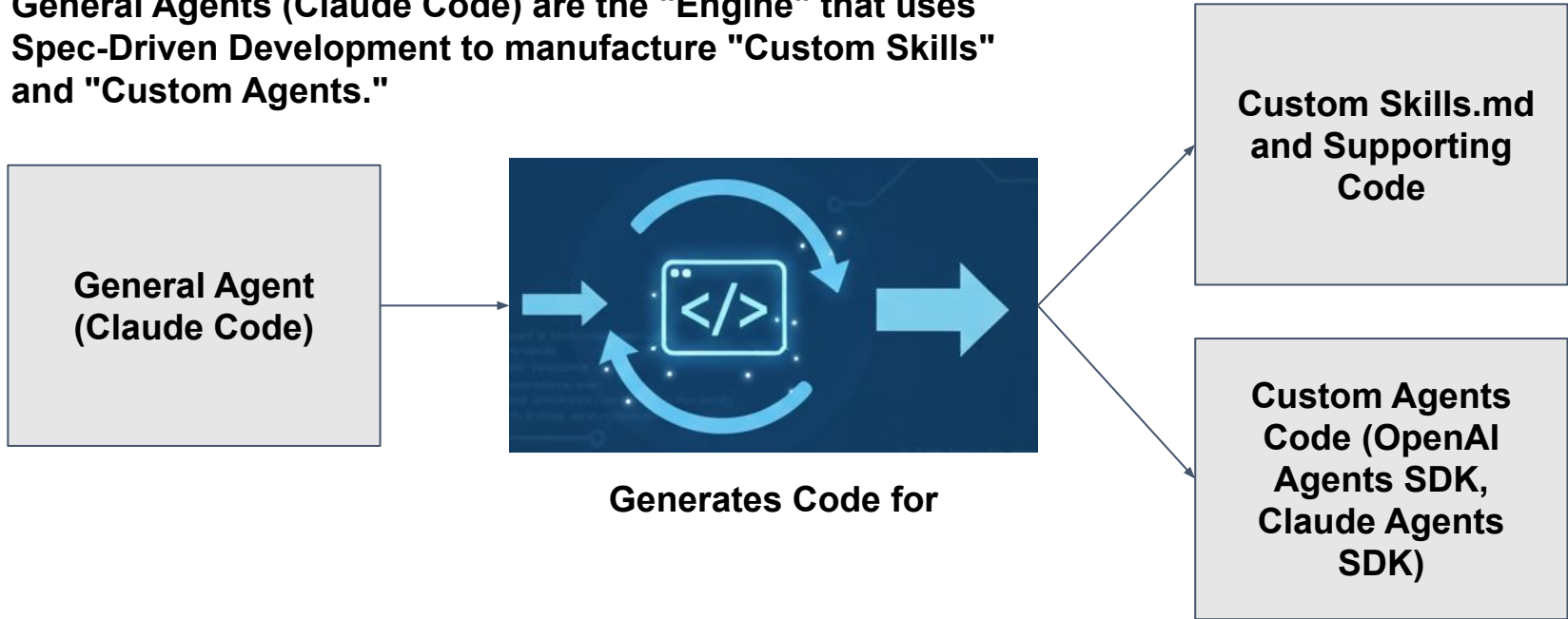
The Workflow

Concept: How the "Factory" works.

1. **The Spec:** You provide a Markdown file describing the goal (e.g., "Automate Q3 Financial Audits").
2. **The Builder (Claude Code):** Analyzes the spec, scans documentation (SDKs/APIs), and identifies the necessary tools.
3. **The Manufacturing:** Claude Code generates Custom Agent or the Custom **SKILL.md** and the supporting code.
4. **The Result:** A production-ready **Custom Agent** or **Custom Skill** is born in minutes.

The Engine of Automated Automation

General Agents (Claude Code) are the "Engine" that uses Spec-Driven Development to manufacture "Custom Skills" and "Custom Agents."



Code: The Universal Interface

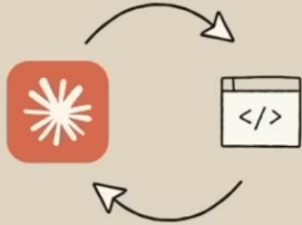
The Technical Engine (Code as the Universal Interface)

Concept: Why a General Agent can build a Custom Agent.

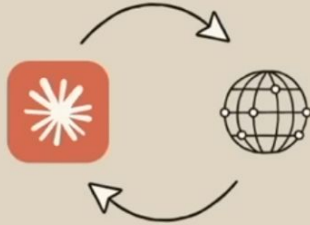
- **The Logic:** AI agents have transitioned from "Text Generators" to "Action Engines."
- **The Loop:** **Claude Code** operates in a **Read-Think-Code-Execute** loop.
- **The Outcome:** Because Code is the universal interface, the General Agent can "write" the instructions ([SKILL.md](#)) or the Python code (OpenAI/Claude SDK) **required for any business use case—from data analytics to document processing.**

How we used to think about agents

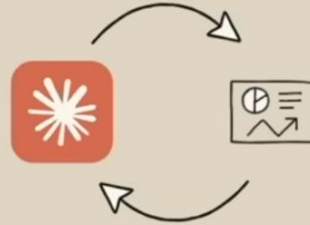
Coding Agent



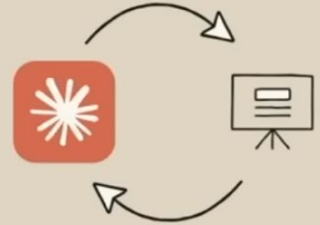
Research Agent



Finance Agent

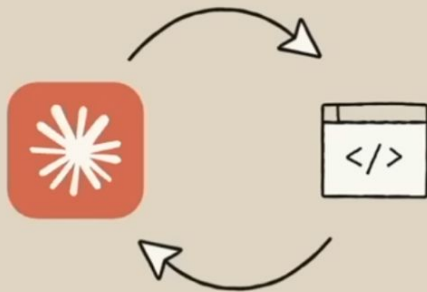


Marketing Agent



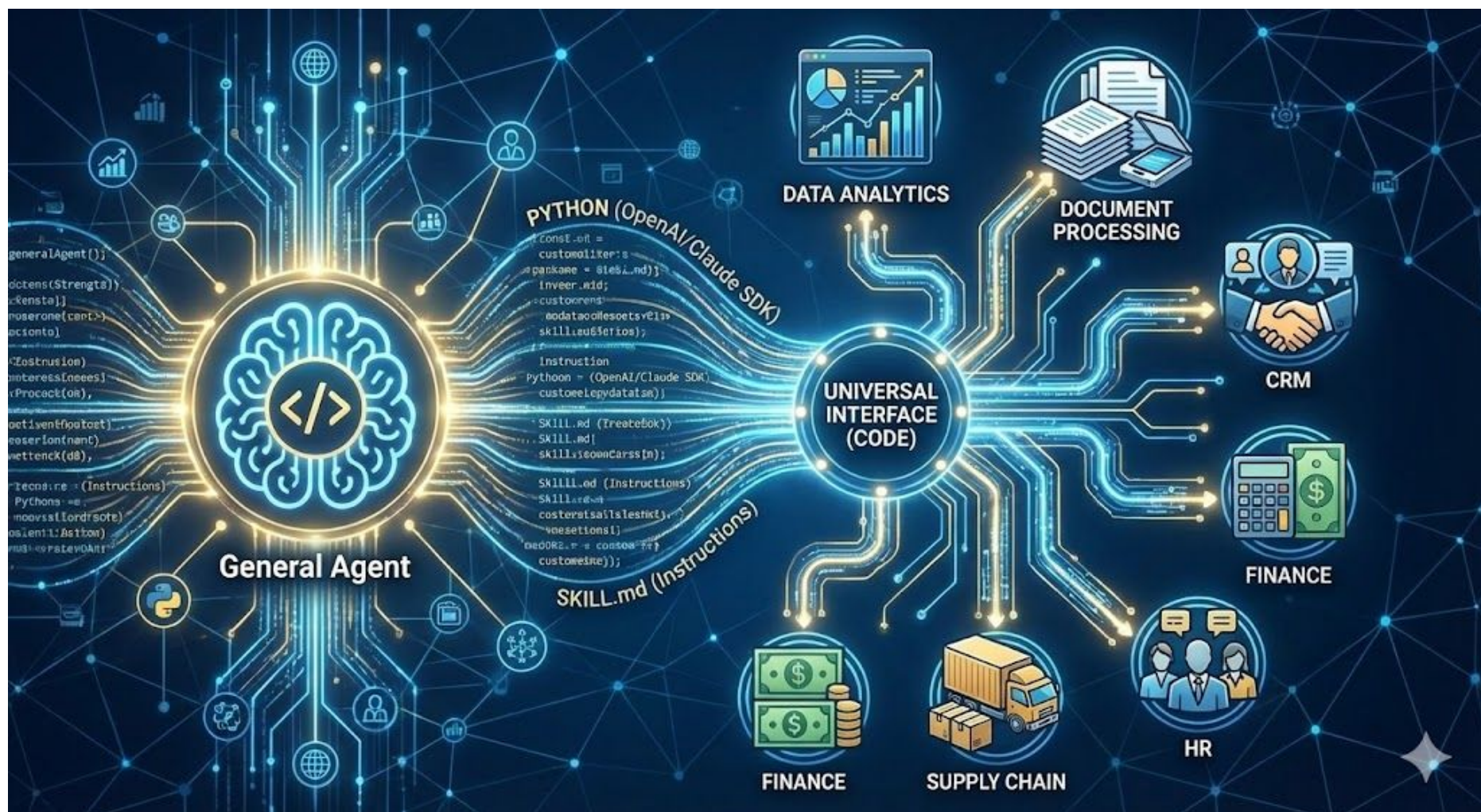
Code is the universal interface

Coding Agent



Use cases





What are Agent Skills?

Skills are **organized collections of files** that package **composable procedural knowledge** for agents

```
anthropic_brand/  
├─ SKILL.md  
├─ docs.md  
├─ slide-decks.md  
└─ apply_template.py
```

Here Trinity (Agent) in Matrix Movie is learning / loading B-212 Helicopter Flying program (Skill, predefined reusable intelligence) in its context, when required.



Skills are just folders

```
anthropic_brand/  
├ SKILL.md  
├ docs.md  
├ slide-decks.md  
└ apply_template.py
```

Skills can include scripts as tools

anthropic/brand_styling/slide-decks.md

```
## Anthropic Slide Decks

- Intro/outro slides
  - background color: `#141413`
  - foreground color: oat
- Section slides:
  - background color: `#da7857`
  - foreground color: `#141413`
```

Use the `./apply\_template.py` script to update a pptx file in-place.



anthropic/brand_styling/apply_template.py

```
import sys
from pptx import Presentation

if len(sys.argv) != 2:
    print("USAGE: apply_template.py <pptx>")
    sys.exit(1)

prs = Presentation(sys.argv[1])
for slide in prs.slides:
    ...
```

Skills are progressively disclosed

anthropic/brand_styling/SKILL.md

```
---  
name: Anthropic Brand Style Guidelines  
description: Anthropic's official brand colors and typography_  
---
```

YAML Frontmatter

Overview

Markdown

This skill provides Anthropic's official brand identity resources for PowerPoint presentations. It includes a pre-branded template and tools to apply Anthropic styling to existing presentations.

Colors

- Dark: `#141413` - Primary text and dark backgrounds
- Light: `#faf9f5` - Light backgrounds and text on dark
- Light Gray: `#e8e6dc` - Subtle backgrounds

Workflows

When creating presentations, read `./slide-decks.md`
When creating professional documents, read `./docs.md`

anthropic/brand_styling/slide-decks.md

Anthropic Slide Decks

- Intro/outro slides
 - background color: `#141413`
 - foreground color: oat
- Section slides:
 - background color: `#da7857`
 - foreground color: `#141413`

... and so on ...

anthropic/brand_styling/docs.md

Documents

- * every document should start with a title, a list of authors, and the creation date
- * if you use tabs in GDocs, make sure the main doc is titled as such

... and so on ...

Manual Prompting vs. Agent Skills (The Evolution)

Concept: Proving the value proposition

Feature	Manual Prompting	Agent Skills (SKILL.md)
Reliability	Ad-hoc / Best effort	Deterministic / Script-backed
Token Cost	Pay for "rules" in every turn	Load rules only when triggered
Asset Type	Disposable conversation	Reusable, scalable IP (Intellectual Property)
Integration	Requires a human "paster"	API-ready via Agent SDKs

Trends in the Skills ecosystem



More complex, production-grade skills



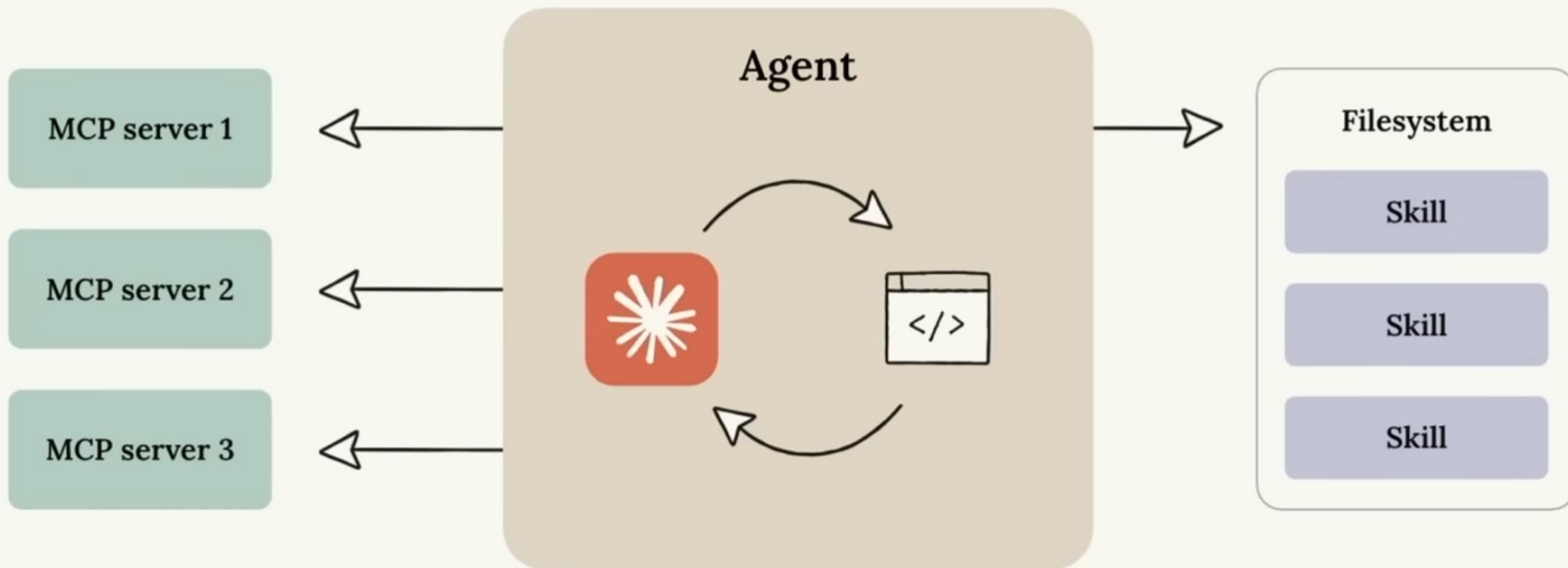
Skills complementing MCP servers



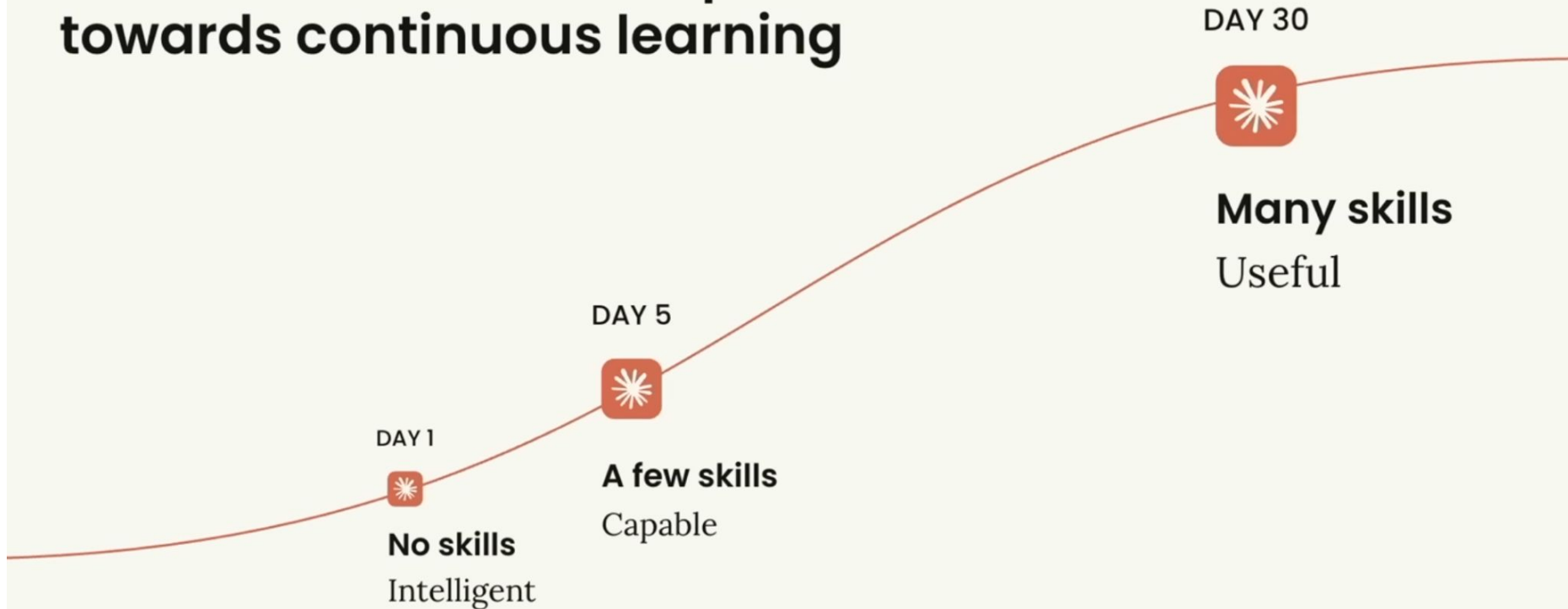
Non-developers building high-value skills

General Purpose Agents

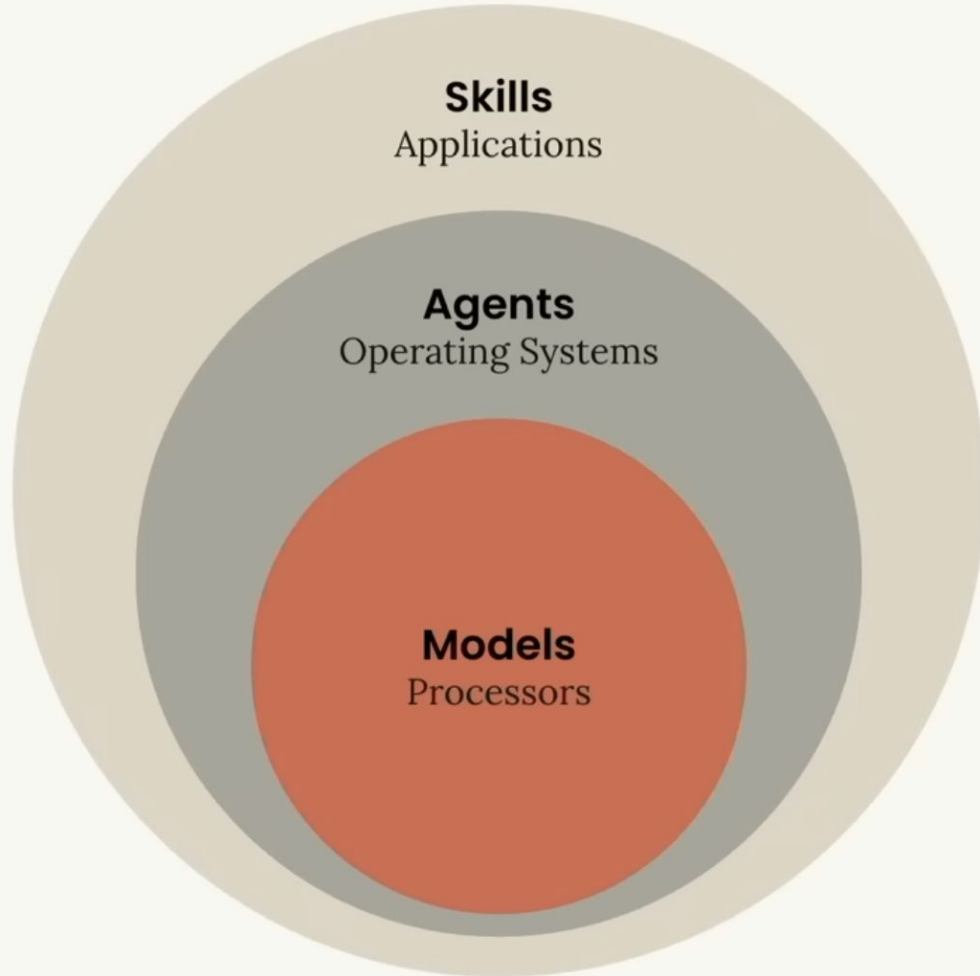
Skills: the complete picture



Skills are a concrete step towards continuous learning



Moving up the stack



Agent Skills: Official Open Standard

Announced December 19, 2025 | agentskills.io

The Announcement



Released as
Open Standard

"We're launching Agent Skills as an independent open standard with a specification and reference SDK" — Mahesh Murag, Anthropic

agentskills.io | github.com/agentskills/agentskills

Official Adopters Grid

Native



Claude Code



Claude.ai



VS Code



GitHub

Adopted



Cursor



Goose



Amp



OpenCode



Letta

💡 Any agent can integrate Skills via agentskills.io/integrate-skills
Same playbook as MCP — open standard, universal adoption

Enterprise
Partners



Notion

ramp



SENTRY



Agent Skills Ecosystem - Industry Adoption (December 2025)

Agent	Vendor	Skills Support	Directory Format	Status
Claude Code	Anthropic	✔ Native	.claude/skills/SKILL.md	Production (Oct 2025)
Codex CLI	OpenAI	✔ Beta	.codex/skills/SKILL.md	Beta (Dec 2025)
Goose	Block (Square)	✔ Adopted	.claude/skills/ + .goose/skills/	Production (2025)
Gemini CLI	Google	🔄 Under Review	—	Issue #12890
Qwen Code	Alibaba	🔄 P1 Roadmap	—	Issue #965



Claude Skills format is becoming the industry standard. Skills written once work across multiple agents.

SKILL.md Format Specification

```
---
name: skill-name
description: One-line description for agent discovery
---
```

Skill Title

When to Use
Trigger conditions and use cases.

Instructions
Step-by-step guidance for the agent.

Examples
Concrete examples of correct behavior.

References
See [REFERENCE.md](./REFERENCE.md) for detailed docs.
See [EXAMPLES.md](./EXAMPLES.md) for more examples.

~100 tokens

Progressive Disclosure

```
.claude/skills/my-skill/
├─ SKILL.md      # Entry point (~100 tokens at startup)
├─ REFERENCE.md  # Loaded on-demand (0 tokens until needed)
├─ EXAMPLES.md   # Loaded on-demand (0 tokens until needed)
└─ scripts/
   └─ helper.py  # EXECUTED, never loaded (0 tokens always)
```

0 tokens (executed only)

SkillPort: Universal Skills for Any Agent

One MCP Server → Skills Everywhere

How It Works

> Install SkillPort

```
pip install skillport
```

2 Add to your MCP config

```
{  
  "mcpServers": {  
    "skillport": {...}  
  }  
}
```

✓ Your agent now has Skills

Same SKILL.md format as
Claude Code



Skills Directory

.claude/skills/
~/.skillport/

SkillPort MCP

- search_skills()
- load_skill()
- read_skill_file()



OpenAI
Agents SDK



LangChain



CrewAI



AutoGen



Custom
Agent

What You Get

- ✓ Same skills work across ALL agents
- ✓ No code changes to your agent
- ✓ Progressive disclosure (token efficient)
- ✓ Works with any MCP-compatible framework



Native support (Claude Code, Cursor, Goose) = No bridge needed.
Everything else = SkillPort is the answer

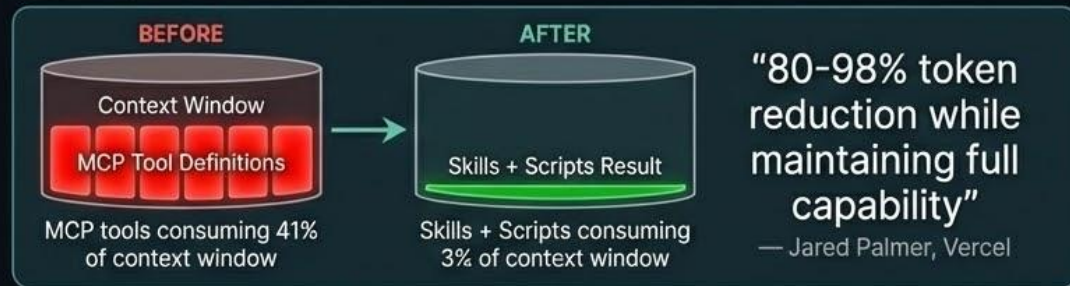
Official spec: agentskills.io/integrate-skills
For tool builders adding native support

MCP + Skills: Token-Efficient Agent Architecture

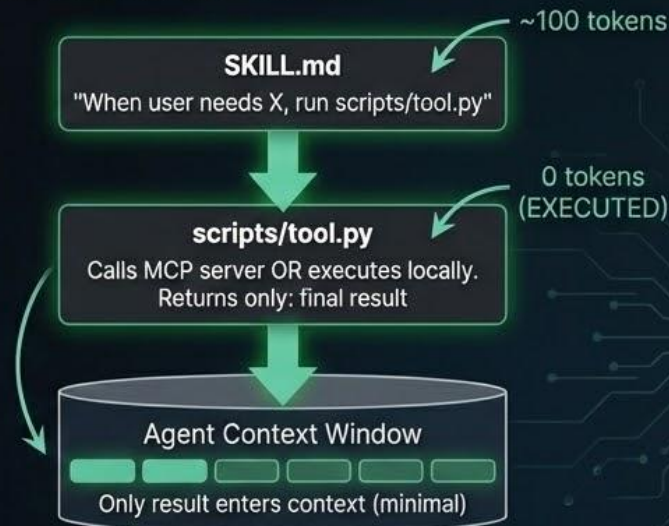
The Token Problem



Result



The Solution Pattern



When to Use Each

Use MCP Directly	Use Skills + Scripts
Simple, single API call	Complex multi-step workflows
Infrequent tool use	Repeated operations
Quick lookups	Data processing, validation

AGENTS.md: Universal Project Instructions

One file → Works with ALL AI coding agents

What is AGENTS.md?

A markdown file that tells ANY AI agent how your project works

- Created by OpenAI
- Adopted by 60,000+ open source projects
- Now an AAIF open standard (Dec 2025)

Works with:



Claude Code



Codex



Goose



Cursor



Copilot



Any Agent

Agent-Specific Files

Agent-specific files (like CLAUDE.md) can include universal instructions by referencing @AGENTS.md:

```
# CLAUDE.md
@AGENTS.md
```

```
## Claude-Specific Features
```

- Tool permissions
- Extended context

```
...
```

Write universal guidelines once in **AGENTS.md**
Reference with **@AGENTS.md** in any agent-specific file

Best Practice

```
your-project/
├── AGENTS.md      # Universal foundation (all agents)
├── CLAUDE.md     # @AGENTS.md + Claude extras (optional)
├── .claude/skills/ # Reusable capabilities
└── src/
```

💡 **AGENTS.md** = Universal project context (write once)
@AGENTS.md = Reference it from any agent-specific file
Works everywhere. No duplication.



Agentic AI
Foundation



THE
LINUX
FOUNDATION



goose

AGENTS.md

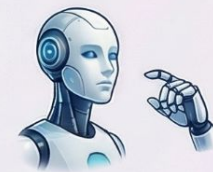
Your Journey to Becoming a Certified Agentic AI & Robotics Engineer

Transform from Code Writer to AI Orchestrator. This certification programme provides a progressive learning path from foundational AI development to advanced humanoid robotics.

LEVEL IV: ENTERING THE PHYSICAL WORLD - HUMANOID ROBOTS

Bridge the Digital-Physical Divide

Give your AI agents a physical body and the ability to operate in reality.



Command the Robotics Stack

Use ROS 2 for control, and Gazebo X-World for advanced simulation.

Create Conversational Robots

Integrate LLMs to build humanoids that can see, speak, and act on commands.

LEVEL III: SCALING SYSTEMS - CLOUD-NATIVE & EVENT-DRIVEN AI

Deploy at Planetary Scale

Use Kubernetes and Dapr to orchestrate hundreds of thousands of agents in the cloud.



LEVEL II: BUILDING INTELLIGENCE - AI AGENT & APPLICATION DEVELOPMENT

Construct Autonomous Agents

Build agents that can plan, remember (using RAG), and use tools to interact.



LEVEL I: THE FOUNDATION AI-DRIVEN & SPEC-DRIVEN DEVELOPMENT



Master AI-Driven Development

Learn to collaborate with AI coding agents like Claude, Gemini, and GPT-5.1.



Think in Specifications

Use Specification-Driven Development (SDD) to create executable blueprints for generating code and tests.



Build with Modern Python

Master asynchronous and type-safe Python tailored for managing complex agentic workflows.



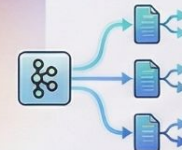
Integrate Complex Data Layers

Combine Vector, Relational, and Graph databases for comprehensive agent memory and knowledge.



Architect End-to-End AI Applications

Secure and containerise full-stack, multi-agent systems ready for deployment.



Enable Asynchronous Communication

Implement event-driven architectures with Apache Kafka for high-throughput, decoupled systems.



Master Production Operations

Learn AIOps for managing observability, security, and CI/CD pipelines for agentic systems.

The AI Revolution in Software

Three parallel paths transforming how we build and deploy technology

Use General Agent to Program (Claude Code)

Supercharge Coding (also solve business problem as General Agent)

Develop AI Agents (Open Agents SDK)

Create Products with AI at the core

AI Ops (Using AI Agents for AI Operations)

Maintain, Monitor, & Scale AI Systems

The AI Development Revolution

The most significant transformation in software development

The scale of transformation

The **\$3 trillion developer economy** (equivalent to France's GDP) and why it's being restructured in 2-3 years instead of the typical 10-15 year cycle.

Developer evolving role

The shift from **developer-as-typist** to **developer-as-orchestrator**.

Why this is different?

Internal disruption (**software disrupting itself**), universal impact (all roles affected), unprecedented speed, and the recursion effect.

The autonomous agent era

The Evolution from code completion → function generation → feature implementation → autonomous agents (Gen 1 through Gen 4).

Essential Tools Ecosystem

The modern AI development toolkit

Coding Agents

- Claude Code
- Goose, Gemini CLI
- OpenAI GPT5-Codex
- Supporting Standards: MCP and Agent Skills

Spec-Driven Development

- Panavesity Spec-Kit Plus
- Amazon Kiro
- Wessl

AI Frameworks

- OpenAI Agents SDK
- Anthropic Agents SDK
- Supporting Standards: MCP and Agent Skills

Deployment

- Vercel / Netlify
- Docker / Kubernetes / Dapr
- Ray

Spec-Driven Development - the future of building digital products

99x

Spec-Driven Development



Spec-Driven Development

Clear specifications unlock AI agent potential

The Problem with "Vibe Coding"

Unclear requirements lead to endless iterations and unpredictable outputs from AI agents

The Solution

Write detailed specs before coding. AI agents execute better with clear instructions.

Benefits

- Consistent AI outputs
- Fewer iterations needed
- Better team alignment

Spec First → AI Executes → Quality Results

Spec Kit Plus

Structured specs for AI agents

What It Provides

Templates and standards for writing clear, actionable specifications that AI agents can execute

Core Components

- Feature specifications
- Vertical Sub agents and Skills (Skill Libraries)
- Prompt History and Architecture decision records
- Test Driven Development

Transform ideas into AI-executable specs

<https://github.com/panaversity/spec-kit-plus>

SPEC.md

Feature: User Auth

Requirements:

- OAuth 2.0
- JWT tokens
- Session mgmt

Acceptance:

- Login < 2sec
- 99.9% uptime

Production Code

→ **AI Agent** →

Spec Driven Development



The Nine Pillars of Agent Factory

1. **AI CLI & Coding Agents** (tools like Claude Code, Gemini CLI, OpenAI GPT5-Codex)
2. **Markdown as Programming Language** (natural language specifications become executable)
3. **MCP Standard** (Model Context Protocol—universal tool integration)
4. **AI-First IDEs** (editors like Zed and Cursor built for AI collaboration)
5. **Linux Universal Dev Environment** (standardized development through WSL/Mac/Linux)
6. **Test-Driven Development** (TDD for quality confidence at scale) also **Evals** for Reasoning Testing
7. **Specification-Driven Development with SpecKit Plus** (structured methodology)
8. **Composable Vertical Skills** (reusable domain expertise components)
9. **Universal Cloud-Native Deployment** (standardized infrastructure with Kubernetes, Docker, Dapr)

THE TOOLING & INTERFACE LAYER



THE LANGUAGE OF SPECIFICATION



ENGINEERING & QUALITY STANDARDS



INFRASTRUCTURE & ENVIRONMENT



Digital FTEs



Agent Evals (The "Exam" for Your Digital Employee)

Goal: To differentiate "Testing Code" (TDD) from "Testing Reasoning."

Enterprises need to know the accuracy rate of an agent before paying for it.

- **The "Golden Dataset":** Before deployment, the agent must pass a set of 50 real-world scenarios (e.g., "Here is a messy invoice, extract the tax ID").
- **Accuracy Scoring:** Move beyond "Pass/Fail" code tests. Introduce "Semantic Similarity" scoring—did the agent understand the *intent* even if the phrasing was different?
- **Regression Testing:** "Every time we update the `SKILL.md`, we run the 'Exam' to ensure previous skills haven't degraded."

Monetizing Expertise: The 2026 Revenue Playbook

Beyond the Service: 4 Models for Monetizing Agent Skills and Custom Agents.

Aligning Price with the Economic Value of Codified Intelligence

Model 1: The "Digital FTE" (Subscription)

- How: Monthly fee for a fully managed, hosted agent (e.g., \$1k/mo).
- Value: "Hands-off" automation for the client.

Model 3: The "License" (IP Ownership)

- How: Annual or Perpetual fee to use your proprietary Agent Skills and logic folders within their infrastructure.
- Value: Client keeps data in-house; you monetize the "Recipe" (The SKILL.md). (Defense, FinTech, Healthcare)

Model 2: The "Success Fee" (Outcome-Based)

- How: Commission on results (e.g., \$5 per lead, 2% of savings).
- Value: High-trust "Pay-per-Value" alignment.

Model 4: The "Skill Marketplace" (Distribution)

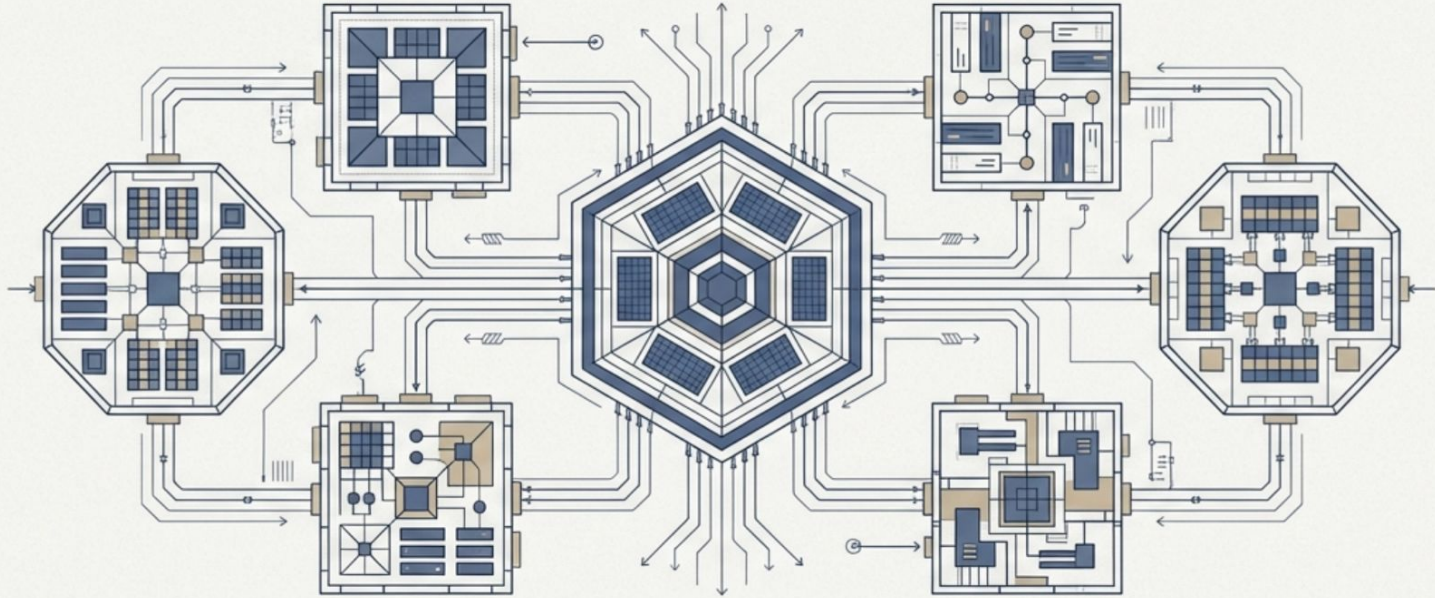
- How: Sell modular "Expertise Packs" via a store or ecosystem.
- Value: Scale through volume; build a brand around specific niche expertise.

Deep Dive into the License Model

In the world of Spec-Driven Development, the "License" isn't just for software—it's for the Skill Folder and Agents

License Type	What is being sold?	Revenue Style
White-Label	The right to rebrand your Agent Skill, MCP or Agents as their own.	High Upfront + Royalty
Enterprise Site License	Unlimited use of a Skill, MCP, and Agents across a whole organization.	Annual Recurring (ARR)
Developer License	The right to use your Skill and MCP as a "sub-module" in their agents or SubAgent.	Usage-based or Flat Tier

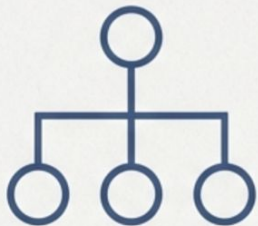
Our Vision



Our vision is to create **Digital Full-Time Equivalents (Digital FTEs)** that perform real, repeatable knowledge work. These are structured AI workers, designed for specific functions, distinct from generic, conversational AI.

What defines a Digital FTE?

A Digital FTE is an AI worker engineered for predictability and purpose. It has four defining characteristics:



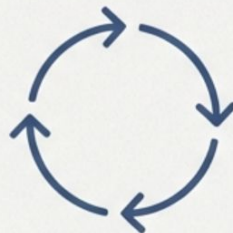
Defined Role

It is assigned a specific job function, similar to a human employee.



Uses Tools & APIs

It interacts with other systems and data sources to complete tasks.



Operates Continuously

It is designed for persistent, ongoing work, not just on-demand queries.



Predictable Cost & Behaviour

Its operational parameters are controlled and measurable.

Digital FTE stands for Digital Full-Time Equivalent

To understand why this is a "Monetization Playbook" item, it's best to look at it as the transition from selling software to selling labor.

1. The Origin: What is an "FTE"?

In traditional business, an **FTE (Full-Time Equivalent)** is a unit of measurement used to represent the workload of one full-time employee (typically 40 hours a week).

- If you have two part-time employees working 20 hours each, they equal **1.0 FTE**.
- Companies use this metric to decide their "headcount" budget.

2. The Shift: What is a "Digital FTE"?

A **Digital FTE** is an AI agent that is built, "hired," and priced as if it were a human employee. Instead of charging for "software licenses" or "API tokens," you are charging for a **virtual role**.

- **Traditional SaaS:** You pay \$50/month per user for a tool (like Salesforce).
- **Digital FTE:** You pay \$1,500/month for an "AI Sales Agent" that performs the work of a junior staffer.

Digital FTE Continued

3. Why this is a Monetization Power-Move

"Digital FTE" is a key model because it changes **who** pays for the AI:

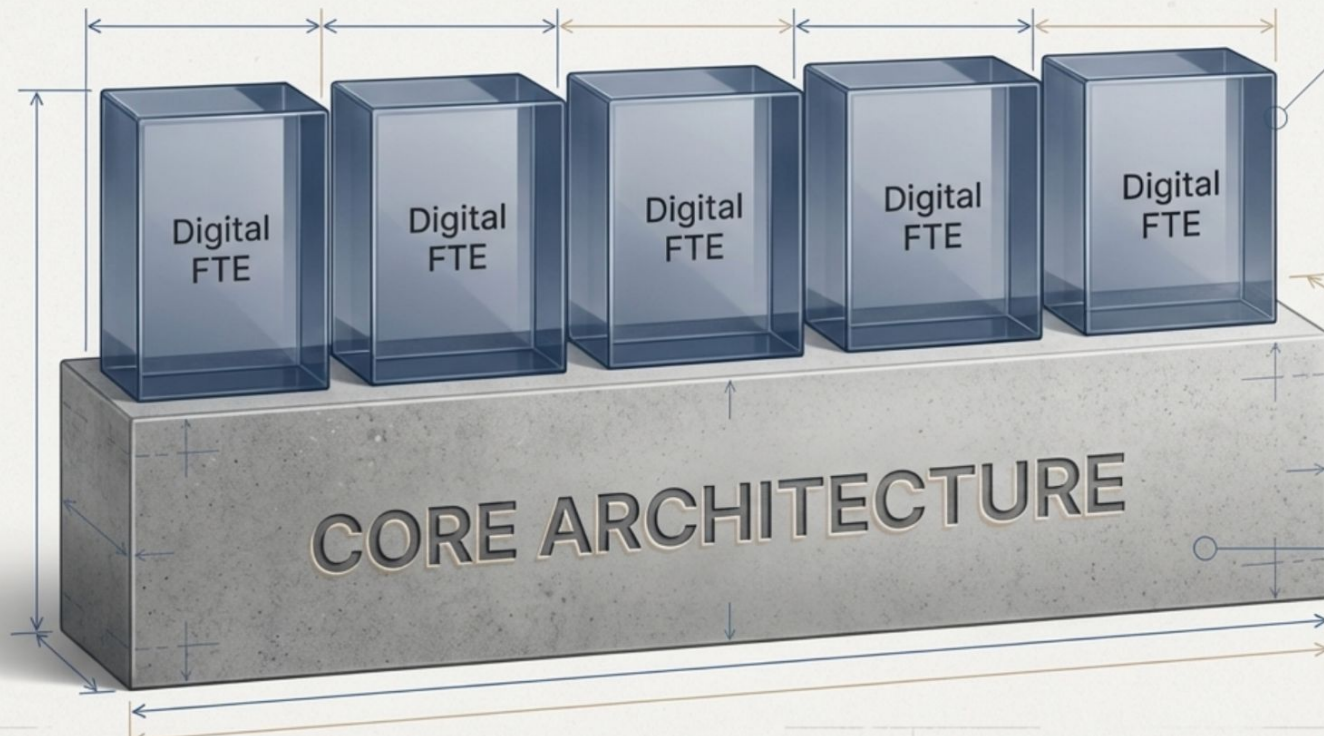
- **Tapping into "Headcount" Budgets:** IT budgets (software) are usually small and strict. HR/Departmental budgets (salaries) are often 10x larger. By calling your agent a "Digital FTE," you can charge significantly more because you are being compared to a **\$50,000 salary**, not a **\$50 software subscription**.
- **Outcome over Usage:** Customers don't care how many "tokens" the agent uses; they care that the job is done. A Digital FTE price is easy for a CEO to understand: *"I'm paying \$1k for a bot that does \$4k worth of human work."*

4. How it fits your "Agent Skills" Story

Using the logic we developed for our previous slides:

1. **Your Knowledge:** You know how to do "Invoice Reconciliation."
2. **Claude Code:** You use it to turn that knowledge into an **Agent Skill** (SKILL.md).
3. **The Digital FTE:** You wrap that skill in a bot and sell it as a **"Digital Accountant" (1.0 Digital FTE)**.

Our strategy is built on a deliberate, architecture-first foundation.



Key Concept: We are pursuing an Architecture-First AI Workforce Strategy.

Explanation: Before building individual agents, we designed a robust and scalable framework. This ensures that every Digital FTE is built on a common, cost-effective, and interoperable platform, preventing the creation of disconnected, expensive AI silos.

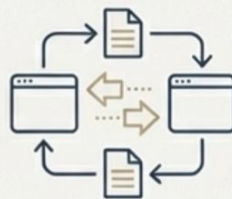
The technology stack is engineered for intelligence and interoperability.

Four core pillars support our Digital FTE ecosystem:



Agent Intelligence

Claude Code with
Agent Skills



Interoperability

MCP (Model Context
Protocol)



Backends

FastAPI + Databases



Infrastructure

Cloud-native

The ROI of Autonomy: Human FTE vs. Digital FTE

Digital FTE is "The 24/7 Employee." Unlike a human 1.0 FTE who works 40 hours, a Digital FTE works 168 hours a week (24/7) with zero fatigue

Feature	Human FTE (Average)	Digital FTE (AI Agent)
Availability	40 hours / week	168 hours / week (24/7)
Monthly Cost	\$4,000 – \$8,000+	\$500 – \$2,000
Ramp-up Time	3 – 6 Months	Instant (via SKILL.md)
Consistency	Variable (85–95% accuracy)	Predictable (99%+ consistency)
Scaling	Linear (Hire 10 people for 10x work)	Exponential (Instant duplication)
Cost per Task	~\$3.00 – \$6.00	~\$0.25 – \$0.50

The 'Aha!' Moment for Your Clients

- A Human FTE works about 2,000 hours a year; a Digital FTE works nearly 9,000
- When you sell a 'Digital Accountant' based on your proprietary knowledge, you aren't just giving them a tool that is 10x cheaper
- You are giving them an employee that never sleeps, never forgets a compliance rule, and can be 'cloned' instantly when the business grows"

Pro-Tip for Your Sales Presentation

Point out that the "Cost per Task" reduction (from ~\$5.00 to ~\$0.50) is an 85–90% cost saving. This is usually the threshold where a CEO will approve a project without further debate.

The "Skill-First" Monetization Pipeline

Strategy Summary: Turning Knowledge into Gold

1. Phase 1: Knowledge Extraction

- Use **Claude Code** + your business "Spec" to build a specialized Skill folder.

2. Phase 2: Asset Hardening

- Finalize the **SKILL.md** and verify the deterministic logic (Python/Bash).

3. Phase 3: Deployment & Capture

- Deploy via the **Claude Agent SDK** or **OpenAI Agents SDK** and choose your monetization pillar (FTE, Success, License, or Marketplace).

The AI Agent Factory: Your Playbook for Monetizing Expertise

The New AI Toolkit: Choosing Your Agent



General Agents: The "Smart Consultant"

Highly flexible AI that autonomously figures out complex, non-routine problems.



Custom Agents: The "Assembly Line"

Purpose-built AI for reliable, specific workflows like customer support or data processing.

Quick Decision Guide

Choose General Agent If...

- **Task Type:** Task is novel or requires problem-solving.
- **End User:** User is technical (e.g., developers).
- **Error Tolerance:** High (a human is in the loop).

Choose Custom Agent If...

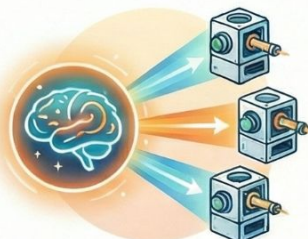
- **Task Type:** Task is repetitive and standardised.
- **End User:** User is non-technical (e.g., customers).
- **Error Tolerance:** Low (must be highly reliable).

The Agent Factory: From Specification to Sale



Step 1: Start with a "Spec"

Write a detailed plan in plain English; this specification becomes your source code.



Step 2: Use a General Agent to Build

A General Agent reads your spec to automatically create custom agents or skills.



Step 3: Sell as a "Digital Full-Time Employee" (FTE)

Price your agent like a virtual employee to highlight its value and massive ROI.

Business Case: Human vs. Digital FTE

	Human FTE (Average)	Digital FTE (AI Agent)
Availability	40 hours / week	168 hours / week (24/7)
Monthly Cost	£3,000 - £6,000+	£400 - £1,500
Scaling	Linear (hire more people)	Exponential (instant duplication)

Case Study: The "Digital SDR" Agent

How a B2B SaaS Startup Replaced a \$100k Headcount with a \$500/mo Agent Skill

The Challenge: A startup with 5,000 monthly leads was only reaching 15% due to human bandwidth. High churn in SDR (Sales Dev Rep) roles and inconsistent follow-up were killing the pipeline.

The Solution (Spec-Driven Workflow):

1. **The Spec:** The founder wrote a Markdown "Spec" detailing the brand voice, objection-handling rules, and qualifying questions.
2. **The Builder: Claude Code** consumed the spec and generated a custom **Agent Skill** folder.
3. **The Asset:** A `sales-prospector/` folder containing `SKILL.md` (instructions) and a `lead_scorer.py` script.

Digital SDR Case Study Continued

Metric	Human SDR Team	Digital SDR (Agent Skill)
Volume	50 outreaches / day	1,000+ outreaches / day
Response Time	4–6 hours	< 2 minutes
Monthly Cost	~\$8,200 (Salary + Tools)	\$500 (Digital FTE Subscription)
ROI (90 Days)	Negative (Ramping/Training)	300% (Instant deployment)

Case Study: CoCounsel" (Legal Research & Contract Review)

THE CHALLENGE

The "Justice Gap" & Manual Drudgery Lawyers were overwhelmed by volume, charging high hourly rates that 85% of low-income people couldn't afford. High-stakes tasks like "document review" cost clients \$1,000 per contract and took days of human time, limiting access to justice and slowing down business deals

THE SOLUTION

The First AI Legal Assistant Instead of a simple chatbot, they built "CoCounsel"—an agent that performs substantive legal work. It doesn't just "chat"; it plans, researches, and drafts like a human lawyer. It was sold as a "seat" for \$500/month (vs. \$20/month for basic tools), replacing expensive outsourced work

Achieved a 97% pass rate on complex legal evaluations and Acquired for
\$650 Million in cash by Thomson Reuters

The "Agent Factory" Roadmap

Next Steps: Building Your Agent Factory. From Your First Spec to Your First Dollar

Day 1-7: Identify the "Knowledge Gap"

- Find a task that is high-volume but currently requires "human judgment" (e.g., Code Review, Lead Gen, Legal Intake).

Day 8-14: Draft the Spec & Build the Skill

- Use **Claude Code** to generate your first **SKILL.md**.
- Use **MCP** to connect it to your actual business data (CRM, Slack, Database).

Day 15-21: Choose Your Monetization Pillar

- Will you sell this as a **Digital FTE** to a client?
- Will you **License** the Skill folder to an enterprise?

Day 22-30: Deploy & Scale

- Use the **Claude Agent SDK** or **OpenAI Agents SDK** to put your agent into production.

The AI Agent Value Chain: From Manufacture to Monetization

Transforming knowledge into profitable digital assets



MANUFACTURE: The Factory

General Agents (Claude Code, Goose) & Spec-Driven Dev (SDD)

➡ Repo-Wide Intelligence



PRODUCT: The Output

Custom Skills (MCP/SKILL.md),
Custom Agents
(OpenAI/Claude SDK) &
End-to-End Solutions



MONETIZATION: The Market

LinkedIn (ROI Selling),
Freelance Platforms
(Project/Retainer) &
Success Fee Model

Golden Rule

In the era of Agents, **your Spec is your Source Code**. If you can describe the excellence you want, AI can build the agent, skills, and MCP to deliver it for **any domain**

The Blueprint for a Perfect Agent Spec

The Anatomy of a High-Value Spec Subtitle: What to Include to Ensure Claude Code Generates a "Senior-Level" Skill

A "Spec" isn't just a prompt; it's a technical requirement document. Use this 6-point checklist before you run the build command.

1. The Identity (Persona)

- [] **Role:** Define the job title (e.g., "Senior Forensic Accountant").
- [] **Tone:** Specify the communication style (e.g., "Concise, professional, and skeptical of anomalies").

2. The Context (MCP & Data)

- [] **Tool Access:** List the specific **MCP Servers** the agent must use (e.g., "Access `stripe-mcp` for transaction history").
- [] **Knowledge Base:** Point to the specific folders or documentation the agent should "study."

3. The Logic (Deterministic Guardrails)

- [] **Mandatory Steps:** Use "First, Then, Finally" logic.
- [] **The "Never" List:** Hard constraints (e.g., "Never approve a refund over \$500 without a human-in-the-loop").
- [] **External Scripts:** Identify where Python/Bash should handle math or file formatting instead of the LLM.

The Blueprint for a Perfect Agent Spec Continued

4. The Success Trigger (The "Trigger" in SKILL.md)

[] **Keywords:** What specific phrases should make Claude say, "I have a skill for this"?

[] **File Types:** Does this skill activate for .pdf, .csv, or .json?

5. The Output Standard (Standardization)

[] **Template:** Provide a Markdown or JSON schema for the final result.

[] **Reporting:** Define how the agent should notify the user (e.g., "Post a summary to #finance-alerts in Slack").

6. The Error Protocol

[] **Fallback:** What should the agent do if the MCP tool is down or data is missing?

Security and Compliance Framework

Non-negotiables for enterprise AI agent deployment

SECURITY LAYERS

1. DATA ENCRYPTION

AES-256 at rest, TLS 1.3 in transit, key rotation every 90 days

2. ACCESS CONTROL

RBAC/ABAC policies, MFA required, least privilege principle

3. AUDIT LOGGING

Immutable logs, 7-year retention, real-time anomaly detection

4. INPUT VALIDATION

Prompt injection prevention, content filtering, rate limiting

COMPLIANCE

EU AI Act (2025)

High-risk AI requires conformity assessments and human oversight documentation.

Consult legal counsel for jurisdiction-specific compliance requirements.

When NOT to Use AI Agents

Strategic restraint is as important as bold adoption

IRREVERSIBLE HIGH-STAKES DECISIONS

Medical diagnoses, legal judgments, large financial approvals without human review

UNDEFINED SUCCESS CRITERIA

Cannot measure success means cannot validate performance or detect failure

UNSTABLE DATA ENVIRONMENTS

Rapidly changing schemas, poor data quality, or missing audit trails

RELATIONSHIP-CRITICAL INTERACTIONS

Executive communications, crisis management, sensitive HR matters

Decision Checklist

1. Can a human review within SLA?
2. Is the error cost acceptable?
3. Are rollback procedures defined?
4. Is training data representative?

If any is No - pause and redesign

PRO TIP

Start with shadow mode - agent recommends but humans execute.
Graduate after 95%+ accuracy over 30 days.

Goal is sustainable automation, not maximum automation. Strategic restraint protects long-term value.

Common Pitfalls and How to Avoid Them

The top 6 failure modes in AI agent deployment

1. Over-Automating Too Fast

FIX: Start with 1-2 low-risk processes. Prove value before scaling.

2. Ignoring Edge Cases

FIX: Document exceptions upfront. Build escalation paths for every decision branch.

3. No Monitoring or Alerting

FIX: Implement observability from day 1.
Track accuracy, latency, and cost per task.

4. Vendor Lock-In

FIX: Build abstraction layers for portability.
Panaversity is working on it.

5. Underestimating Change Management

FIX: Train affected teams early. Position AI as augmentation, not replacement.

6. No Success Metrics Defined

FIX: Define KPIs before building. Measure: time saved, error rate, cost, satisfaction.

80% of AI agent failures stem from organizational issues, not technical ones. Plan for people first.

Team Structure for AI Implementation

The roles you need for successful AI agent deployment

CORE TEAM (Required)

AI Product Owner

Owens roadmap, prioritizes use cases, defines success metrics

Agent Engineer

Builds skills, prompts, workflows; integrates MCP

Domain Expert

Provides business logic, validates outputs, trains edge cases

EXTENDED TEAM (Scale)

Security/Compliance Lead

Audits, access controls, regulatory alignment

MLOps Engineer

Monitoring, deployment pipelines, cost optimization

Change Manager

Training, adoption tracking, resistance management

Team Size by Stage

POC / Pilot

2-3

Production

4-6

Enterprise Scale

8-12

Rule of thumb: 1 Agent Engineer per 3-5 production agents. Start small with core team.

Competitive Landscape: AI Agent Platforms

Platform selection guide for enterprise deployment (2026)

RECOMMENDATION

Claude Code + Spec Kit Plus for development
→ OpenAI/Claude Agent SDK for production

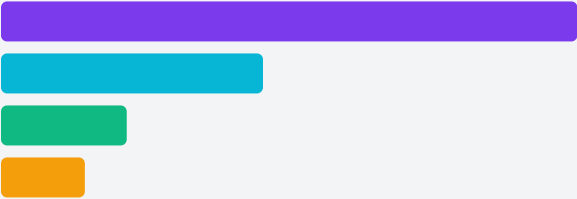
2026 TREND

MCP adoption becoming table stakes. Skills
standardizing across platforms.

Pricing & Total Cost of Ownership

Budget planning guide for AI agent deployments

COST STRUCTURE BREAKDOWN



TOKEN COSTS (December 2025)

Gemini 3 Flash Preview

\$0.5/1M input • \$3/M output

GPT-5.2

\$1.75/400k input • \$14/128k output

Claude Opus 4.5

\$5/1M input • \$25/M output

DeepSeek-V3.2

\$0.28/M input • \$0.42/1M output

MONTHLY BY SCALE

Starter

\$500-2K

1-2 agents, 10K tasks/mo

Growth

\$5-15K

5-10 agents, 100K tasks/mo

Enterprise

\$30-100K

20+ agents, 1M+ tasks/mo

ROI FORMULA

(Hours Saved × Rate) = Savings

Target: 3-6 month payback period

Token costs trending down 30-50% annually. Plan for model upgrades every 6-12 months.

How to Reach the Global Market?

Here are the concluding slides of the presentation,
designed to bridge the gap between building a
product and dominating a global vertical market and
establish Unicorn AI Startups.

The Enterprise North Star: The Digital FTE

The Way Forward: Developing Digital FTEs for the Enterprise: Shifting from "Tools" to "Teammates"

The Objective: Our goal is not just to build software, but to deploy **Digital Full-Time Equivalents (FTEs)** that sit alongside human teams.

The Construction Stack:

- **The Architect: Claude Code** (General Agent) designs the logic.
- **The Framework: OpenAI Agents SDK** provides the custom runtime.
- **The Intelligence: Agent Skills** (**SKILL.md**) provide the expertise.
- **The Connectivity: MCP Servers** provide the real-world data access.

The Outcome: A secure, specialized, and autonomous employee that works 24/7.

"We've seen how to build the brain and the hands. But the true value for an enterprise isn't a new app—it's an FTE. We are building digital teammates that can be deployed instantly into any corporate workflow."

The Scaling Paradox

The Billion-Dollar Question: How Do We Scale? Reaching Millions of Enterprises with Limited Resources

The Barrier: Traditional enterprise sales and infrastructure are slow and expensive.

The Challenge:

1. **Distribution:** How do we put our Digital FTEs in front of decision-makers globally?
2. **Scalability:** How can a small team support 1,000,000+ businesses simultaneously?

The Reality: To build a **Unicorn**, we must decouple our human effort from our global reach.

"To build a billion-dollar company, you must stop hiring people to grow. You need to make sure your company can reach millions of customers without needing a massive team to do the manual work."

The Scaling Paradox

The Billion-Dollar Question: How Do We Scale? Reaching Millions of Enterprises with Limited Resources. How can we scale for every industry—accounting, law, sales, and support.

The Barrier: Traditional enterprise sales and infrastructure are slow and expensive.

The Challenge:

1. **Distribution:** How do we put our Digital FTEs in front of decision-makers globally?
2. **Scalability:** How can a small team support 1,000,000+ businesses simultaneously?

The Reality: To build a **Unicorn**, we must decouple our human effort from our global reach.

"To build a billion-dollar company, you must stop hiring people to grow. You need to make sure your company can reach millions of customers without needing a massive team to do the manual work."

Distribution: The OpenAI Apps Ecosystem (chatgpt.com/apps)

Reaching 800 Million Users via OpenAI Apps. The Global Marketplace for Digital FTEs

Instant Visibility: OpenAI Apps provide a direct pipeline to:

- **800+ Million** individual users.
- **1+ Million** businesses already using the OpenAI ecosystem.

The "App Store" Moment: Just as the App Store created the Mobile Economy, OpenAI Apps is creating the **Agent Economy**.

Low Friction: Enterprises can discover and "hire" your Digital FTE with a single click, bypassing traditional 6-month sales cycles.

"We don't need a sales team of 500 people. We leverage the world's most powerful AI distribution engine. By placing our Custom Agents on the OpenAI platform, we are standing in front of a million businesses on day one."

Reliability: The Cloud Native Backbone

Scaling via Cloud Native Technologies. Designing for Infinite Growth and Enterprise Security.

The Foundation: We use **Cloud Native** (Kubernetes, Docker, Dapr, Serverless) to host our agents.

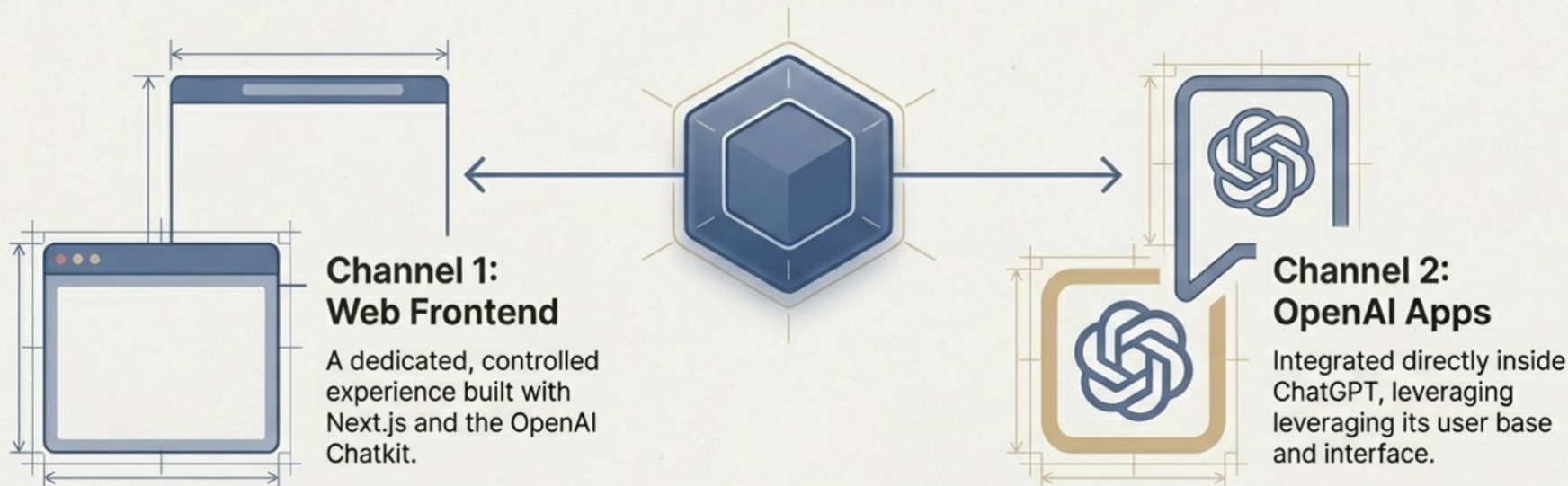
Why Cloud Native?

- **Auto-Scaling:** Your infrastructure grows automatically as you sign up the next 100,000 enterprises.
- **Multi-Tenancy:** Keep enterprise data isolated and secure at a massive scale.
- **High Availability:** Ensure your Digital FTEs never "go home" or experience downtime.

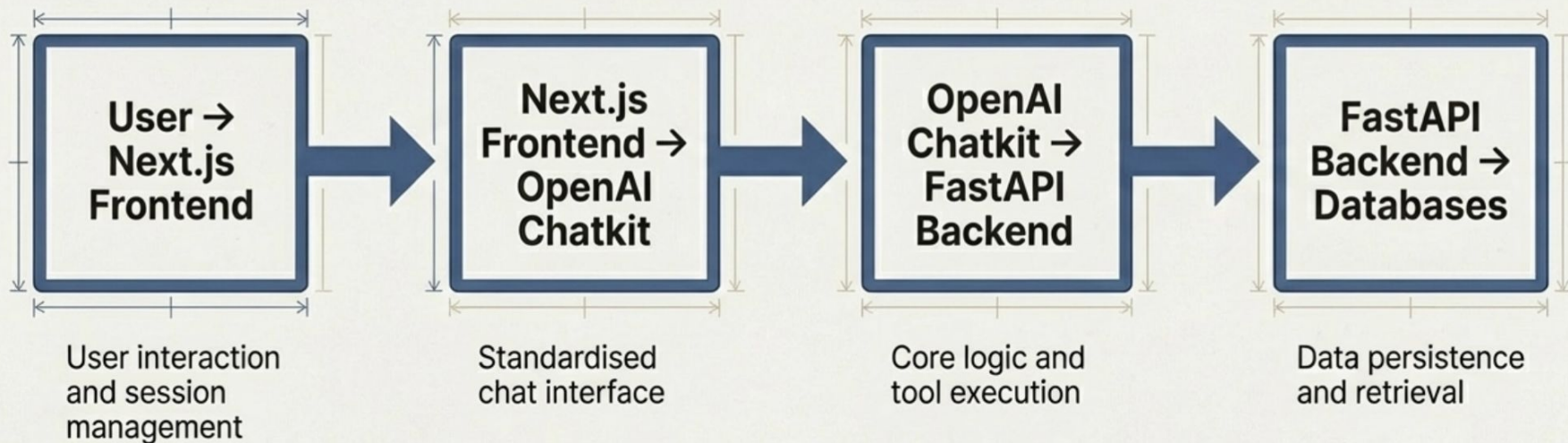
Economic Leverage: Pay only for the compute you use on any cloud (AWS, Google Cloud, Azure, Digital Ocean), keeping margins high.

Our Digital FTEs operate across two primary channels.

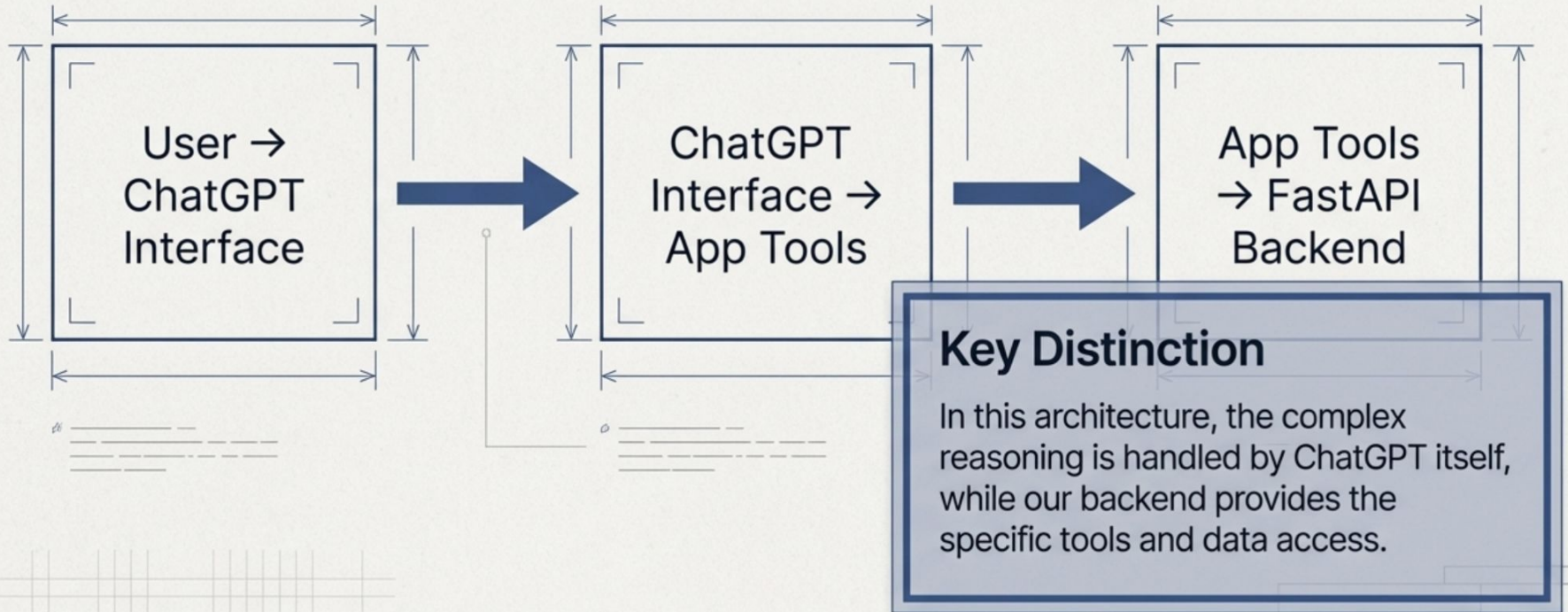
To maximise reach and utility, we have engineered a dual frontend strategy, allowing users to interact with our Digital FTEs via our own platform or within the ChatGPT ecosystem.



The Web Frontend architecture provides a dedicated user experience.



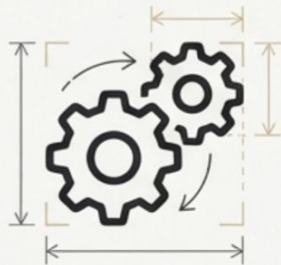
The OpenAI App architecture leverages the reasoning power of ChatGPT.



The engine room: A dual-backend system for flexibility and control.

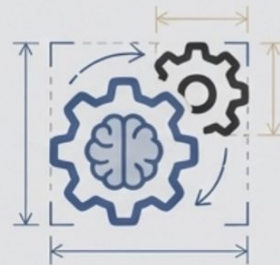
The heart of our architecture is a strategic separation of backend types. This allows us to precisely balance intelligence with operational cost.

Deterministic Backend (Zero LLM)

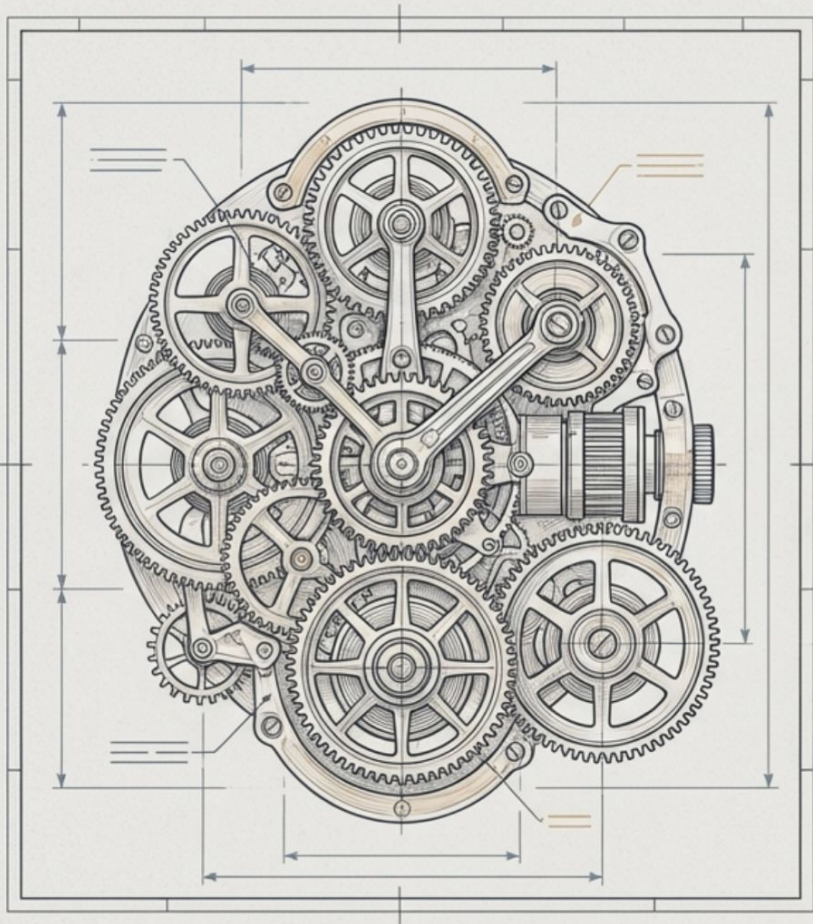


No LLM calls.
Predictable, low cost, high control.

Hybrid Backend



Uses LLM calls.
Higher intelligence, higher cost.

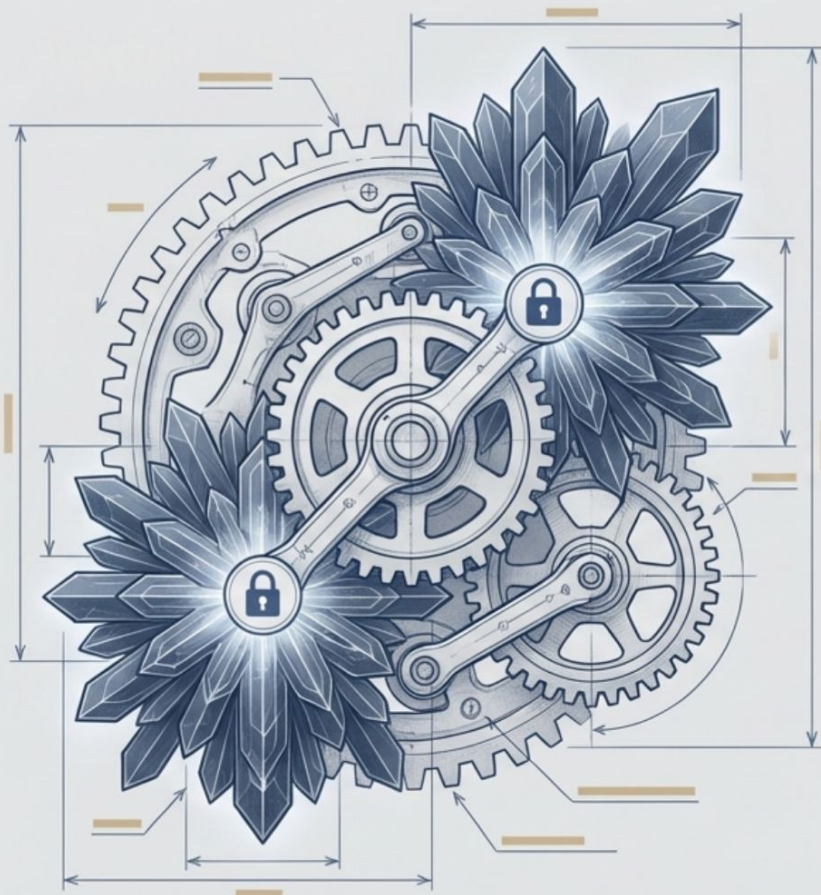


The 'Zero LLM' backend is our foundation for scale and compliance.

This backend processes requests using deterministic logic without making calls to a large language model. All sophisticated reasoning is deliberately kept on the frontend (e.g., in ChatGPT or Claude).

Best Suited For:

- Cost-conscious applications
- High-volume usage scenarios
- Workloads requiring strict compliance and control



The Hybrid Backend is reserved for monetised, premium features.

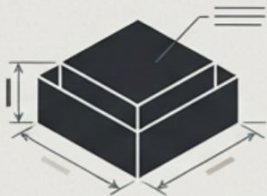
This backend selectively uses LLM calls to perform tasks that require a higher degree of intelligence and nuance.

Used For Premium Features:

- Deep analysis of complex data
- Synthesis of multiple information sources
- Advanced personalisation

The Business Rule: Access to the Hybrid Backend is always gated and monetised.

Our entire architecture is governed by a principle of radical cost discipline.



Zero LLM by default. This is our baseline for all Digital FTEs, ensuring a **low** and **predictable** cost structure.

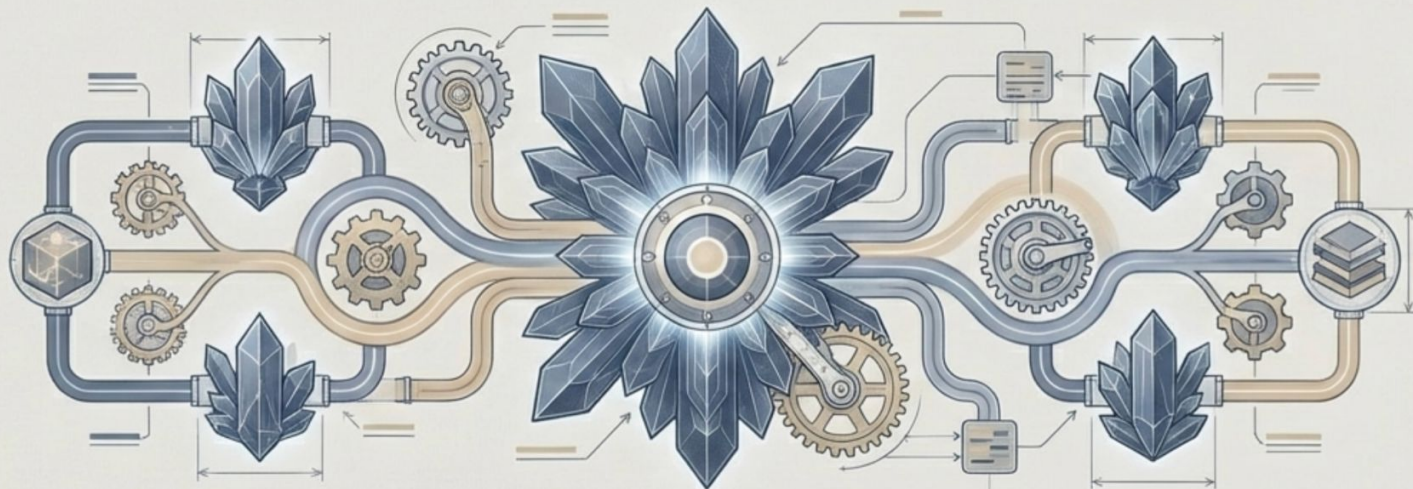


Hybrid LLM usage is a deliberate, **selective** choice, reserved exclusively for **premium**, value-added capabilities.



This disciplined approach enables the creation of truly **sustainable** Digital FTEs, avoiding the unpredictable and often prohibitive costs of unrestrained LLM usage.

The result: A scalable ecosystem of intelligent, cost-effective Digital FTEs.



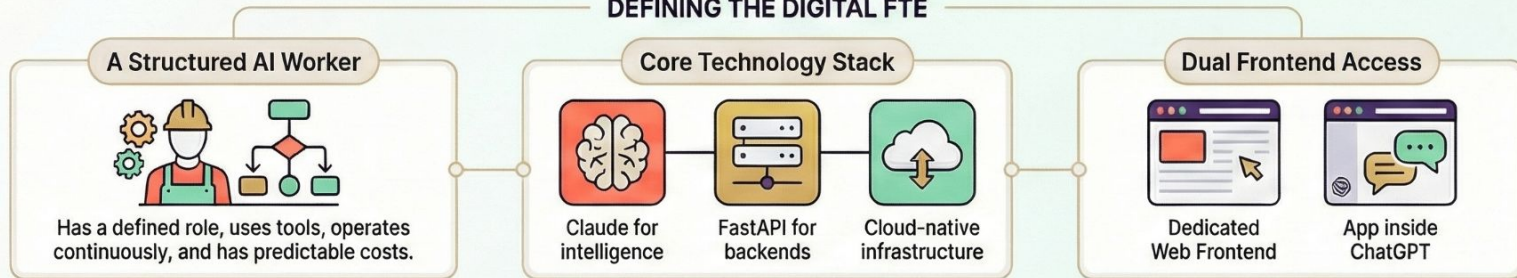
Our architecture-first strategy delivers a new class of AI worker that can:

- ✓ **Augment** or **replace** real-world knowledge roles effectively.
- ✓ Run **seamlessly** across both dedicated web applications and the ChatGPT platform.
- ✓ **Intelligently balance** advanced AI intelligence with strict economic discipline.

The Architecture of a Digital FTE

Building structured, cost-effective AI workers

DEFINING THE DIGITAL FTE

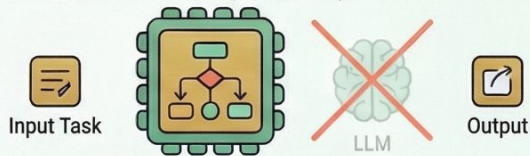


THE CORE STRATEGY: TWO BACKEND ARCHITECTURES

DETERMINISTIC BACKEND (ZERO LLM)

The default choice.

No LLM calls for maximum predictability, control, and low cost.

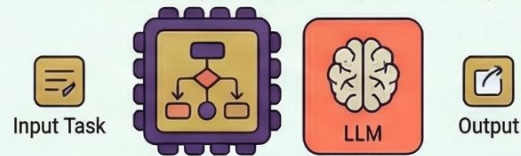


💰	PRIMARY GOAL:	Cost Efficiency & Control
📋	BEST FOR:	No LLM calls for maximum predictability, control, and low cost.
🔒	BUSINESS MODEL:	Default, core functionality

HYBRID BACKEND

The premium choice.

Uses LLM calls selectively for higher intelligence and deep analysis.



🧠	PRIMARY GOAL:	Advanced Intelligence
🔍	BEST FOR:	Uses LLM calls selectively for higher intelligence and deep analysis.
🏠	BUSINESS MODEL:	Gated, monetised premium features

Glossary of Key Terms

Essential vocabulary for AI agent development and deployment

AI Agent

Software that pursues goals autonomously by observing, deciding, acting, and learning in a loop.

SKILL.md

Markdown file containing instructions, logic, and workflows that teach an agent a specific capability.

General Agent

Flexible agent (Claude Code, Goose) that handles diverse tasks with zero-shot planning. Builder tool.

Digital FTE

AI agent priced/sold as equivalent to a full-time employee. Works 168 hrs/week vs human's 40.

Human-in-the-Loop (HITL)

Pattern where humans review/approve agent decisions for high-stakes or edge case scenarios.

Guardrails

Hard constraints defining what an agent can/cannot do. Essential for production safety.

MCP (Model Context Protocol)

Universal protocol for connecting AI agents to external tools, databases, and APIs.

Spec-Driven Development (SDD)

Methodology where detailed specifications are written first, then AI generates code to meet them.

Custom Agent

Purpose-built agent (via SDK) optimized for specific workflows with guardrails. Production tool.

Agent Factory

System using General Agents + Specs to manufacture Custom Skills and Agents at scale.

Shadow Mode

Deployment pattern where agent runs in parallel with humans, outputs compared but not acted upon.

Orchestration

Coordinating multiple agents/skills to complete complex multi-step workflows.

Core Insights

Key takeaways from the research

- **Work will be a partnership of people, agents, and robots**
- **Automation transforms work, doesn't eliminate it**
- **Most human skills remain relevant but evolve**
- **AI fluency is the fastest-growing workforce requirement**
- **Workflow redesign unlocks greater value than task automation**

Final Summary of the Presentation Flow

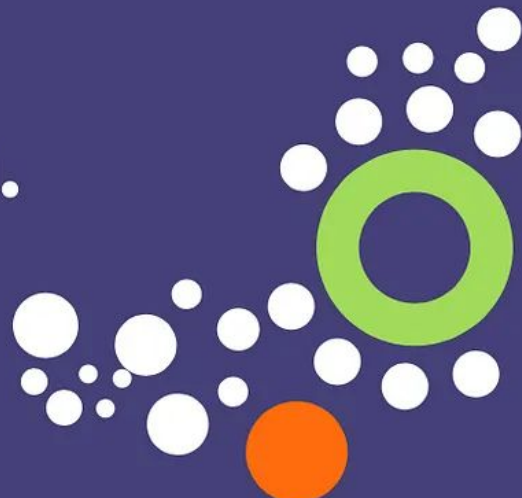
Key takeaways from Presentation

1. **The Rise of Agents:** The landscape is shifting to Agentic workflows.
2. **General vs. Custom:** Use the General (Claude Code) to build the Custom (OpenAI SDK).
3. **The Universal Interface:** Code and MCP are how agents actually "work."
4. **Agent Skills:** The portable **SKILL.md** is your new monetizable asset.
5. **Monetization:** Use the License or Digital FTE models to capture value.
6. **Action Plan:** Start with a Spec, build with Claude, and deploy for ROI.

There comes a time
we need to stop reading the
books of others.

And write our own.

- Albert Einstein





Join the Spec-Driven Revolution

Learn AI-native development with reusable intelligence at

[Panaversity.org](https://panaversity.org)

[PIAIC.org](https://piaic.org)

[Connect Today](#)